# Manual para el curso Mineria de Datos y Análisis Multivariado (XS-0126)

**Autor:** Anderson Martínez
**Colaboradores:** Sebastián Calvo, Justin Zhu, Fabián Almirall

## Contents

**1 Aprendizaje supervisado** **3**

**2 Aprendizaje no supervisado** **4**

**3 Diferencia entre aprendizaje supervisado y no supervisado** **4**

**4 Características del aprendizaje no supervisado** **4**

**5 Proceso de Análisis de componentes principales (PCA)** **6**
5.1 Maldición de la dimensionalidad . . . 7
5.2 Requisitos para que PCA tenga sentido . . . 7
5.3 Ejemplo de aplicación . . . 7

**6 Comparación de análisis factorial con componentes principales** **9**
6.1 Ejemplo de análisis factorial . . . 10
6.2 Diferencias entre análisis factorial y PCA . . . 10

**7 Valores y Vectores propios** **10**
7.1 ¿Cuándo centar y cuándo estandarizar? . . . 11
7.2 Valores y vectores propios en R . . . 11
7.3 Construcción de Z's . . . 13
7.4 Estudio de intervalos en los que se encuentran los datos generados por un componente . . . 14
7.5 Correlación entre Componentes y variables . . . 15

**8 Aplicación de PCA ejemplo de HEPTATLON** **15**
8.1 Proceso de cambios en las variables . . . 15
8.2 Matriz de correlaciones . . . 16
8.2.1 Cálculo en r . . . 16
8.2.2 Gráfico de correlación librería corrgram de colores . . . 16
8.2.3 Gráfico de correlación librería corrgram de números (correlaciones) . . . 17
8.2.4 Gráfico del paquete corrplot . . . 17
8.3 Análisis de valores extremos mendiante leverage . . . 17
8.3.1 Formas de averiguar y eliminar el valor extremo . . . 18
8.4 Obtención del determinante de la matriz R . . . 18
8.5 Hipótesis de matriz de correlaciones . . . 19
8.6 KMO . . . 19
8.7 Análisis con la matriz de covarianzas . . . 20
8.7.1 Comparación de varianzas de la variables . . . 20
8.7.2 Descomposición espectral de matriz de covarianzas . . . 20

8.8 Obtención de los componentes principales $Z$'s . . . 22
8.8.1 Cálculo de cuánto porcentaje explica el componente . . . 22
8.8.2 ¿Es normal que tenga media cero? . . . 22
8.8.3 Forma automática con `prcomp()` . . . 22

**9 Análisis con matriz de correlaciones** **23**
9.1 Estadarizando las variables prrimero . . . 23
9.2 Descomposición espectral de la matriz de correlaciones y obtención de los valores propios. . . . 24
9.3 Porcentaje de explicación de un componente principal . . . 24
9.4 Forma automática . . . 25
9.5 Obtención del primer componente manualmente . . . 25
9.6 Comparación de Varianzas y lambdas . . . 26
9.7 Cambio a los signos de los coeficientes de $Z_1$ . . . 26

**10 Biplot** **26**
10.1 Biplot categórico . . . 27

**11 Gráfico de sedimentación** **28**
11.0.1 Conclusión de estudio . . . 29
11.1 Reproducción de la matriz de correlaciones . . . 29
11.2 Obtención de correlaciones . . . 30

**12 Prueba de los componentes en una regresión** **30**
12.1 Forma automática . . . 31
12.2 Interpretaciones de regresión . . . 31

**13 Conglomerados** **32**
13.1 Distancias entre individuos . . . 32
13.2 Distancia Euclidea . . . 32
13.3 Distancia de Mahalanobis . . . 33
13.3.1 Forma automática . . . 35
13.4 Distancia City-block (Manhattan) . . . 35
13.4.1 Nota importante de las distancias . . . 36

**14 Ejemplo de aplicación con temperaturas de diferentes cantones de Costa Rica** **36**
14.0.1 Cálculo distancia euclidiana . . . 37
14.0.2 ¿Que sucede si transponemos la base? . . . 37
14.0.3 Extración de valores de un objeto dist() . . . 38
14.0.4 Cálculo distancia de Mahalannobis . . . 39
14.0.5 Cálculo distancia de Manhattan . . . 39
14.1 Creación de dendogramas y distancias entre grupos (linkage) . . . 39
14.1.1 K-vecino más cercano (singke linkage) . . . 39
14.1.2 K-vecino más lejano (complete linkage) . . . 40
14.1.3 Salto promedio . . . 41
14.1.4 Distancia de Ward . . . 42
14.2 ¿Los dos cantones más correlacionados son los cantones que menor distancia tienen? . . . 43
14.3 ¿Que ocurre cuando tenemos variables categóricas? . . . 48
14.3.1 Ejemplo de las universidades . . . 48
14.3.2 Ejemplo de universidades aplicado en r . . . 50

**15 Método de k-medias y k-mediodes** **51**
15.1 k-medias . . . 51
15.1.1 Algoritmo 1 . . . 52
15.1.2 Función de SCDG . . . 53
15.1.3 Algoritmo 2 . . . 57

15.1.4 Desventajas . . . 57
15.1.5 Mezcla de dendograma con k-medias . . . 57
15.2 k-medoides . . . 57
15.2.1 Cálculo medoide . . . 58
15.3 Algoritmo PAM (partitioning around medoids) . . . 59
15.3.1 Observaciones sobre k-medoides . . . 59
15.3.2 CLARA (clustering large applications) . . . 59
15.3.3 Formas automáticas . . . 59

**16 Creación de grupos o clusteres** **60**
16.0.1 Comparación de grupos con variables continuas . . . 60
16.0.2 Comparación de grupos con variables categóricas . . . 61
16.0.3 K-medias con PCA, para graficar . . . 62
16.0.4 Combinación de dendograma con gráfico de clúster . . . 62
16.1 Métodos difusos . . . 63
16.1.1 Código para sacar la distancia entre la mayor y menor probabilidad de cada observación 64
16.1.2 ¿Que tán difusos están los datos? . . . 64
16.2 Obtención del número de grupos óptimo mediante el método de la silueta promedio . . . 64
16.2.1 Cluster Jerárquico con Silueta . . . 65
16.3 Estadístico GAP . . . 66
16.3.1 Construcción del estadístico WAP . . . 66
16.4 Validación . . . 66
16.5 Indice Dunn . . . 67
16.5.1 Índice Dunn . . . 67
16.6 Mapas de calor (heatmaps) . . . 68

**17 Escalamiento multidimensional (MDS)** **69**
17.1 Escalamiento multidimensional métrico (MMDS) . . . 69
17.1.1 Datos Curiosos de MMDS . . . 69
17.1.2 Escalamiento Multidimensional en r . . . 71
17.1.3 Reconstrucción por la matriz B . . . 71
17.2 Bondad de ajuste (Mardia et al. 1979) . . . 71
17.2.1 Obtención de valores propios (Eigen values) . . . 72
17.2.2 Aplicación de la bondad de ajuste . . . 72
17.3 Falta de ajuste (stress) . . . 72
17.3.1 Aplicación del strees . . . 73
17.3.2 Comparación de valores de r de coordernadas con los valores propios . . . 73

**18 Códigos extras** **75**
18.1 Centroides nuevos a partir de una base de datos . . . 75
18.2 Evaluar el agrupamiento a partir de la SCDG . . . 76
18.3 K-means de forma manual . . . 76
18.4 Cálculo de medoides aplicado . . . 77
18.5 Código para distancia entre grupos de forma manual a partir de las distancias: . . . 78
18.6 Suma de cuadrados de grupo al medoide . . . 79
18.7 Distancias para patrones similares (correlación) . . . 80

# 1 Aprendizaje supervisado

Se aprenden funciones, relaciones que asocian entradas con salidas, por lo que se ajustan a un conjunto de ejemplos de los que conocemos la relación entre la entrada y la salida deseada.

Dependiendo del tipo de salida, suele darse una subcategoría que diferencia entre:

- Modelos de clasificación, si la salida es un valor categórico (por ejemplo, una enumeración, o un conjunto finito de clases).
- Modelos de regresión, si la salida es un valor de un espacio continuo.

## 2 Aprendizaje no supervisado

No estamos interesados en ajustar pares (entrada, salida), sino en aumentar el conocimiento estructural de los datos disponibles (y posibles datos futuros que provengan del mismo fenómeno), por ejemplo:

- Dando una agrupación de los datos según su similitud.
- Simplificando las estructura de los mismos manteniendo sus características fundamentales (como en los procesos de reducción de la dimensionalidad).
- Extrayendo la estructura interna con la que se distribuyen los datos en su espacio original (aprendizaje topológico).

## 3 Diferencia entre aprendizaje supervisado y no supervisado

El aprendizaje supervisado tiene variable respuesta $Y$, mientras que el aprendizaje no supervisado, no la tiene. Además es importante resaltar que el no supervisado es más descriptivo.

## 4 Características del aprendizaje no supervisado

- No existe una salida o variable respuesta, por lo que no se utiliza un modelo que relacione un conjunto de variables con una respuesta.
- Esto hace que sea difícil validar los resultados, al contrario del aprendizaje supervisado en el que existen métricas que permiten evaluar con claridad la bondad del aprendizaje realizado.
- Es parte de un análisis exploratorio de datos.
- Algoritmos no supervisados resultan ser muy costosos porque requieren de más pruebas de ensayo y error, haciendo que requieran de un aparataje teórico y computacional mucho más elaborado.

**Ejemplo 1**

Suponga que tiene la siguiente regresión:

$$\mu_{y|x} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_{p-1} X_{p-1}$$

Recuerde que $\mu_{y|x}$ es la respuesta

Se tiene el siguiente subconjunto der X's:

$$X_1, \ldots, X_k \text{ Altamente Correlacionadas}$$

**Nota:** Cuando las X's están muy correlacionadas hay problemas de multicolinealidad y eso nos infla los errores estándar.

Note que este conjunto de variable es de dimensión $k$, esto lo que nos dice en realidad es que estas X's miden un subconjunto menor de conceptos, lo quye nos lleva al siguiente conjunto:

$$Z_1, \ldots, Z_q$$

Donde $q < k$ (No Correlacionadas)

Reemplanzado en el modelo original tenemos el siguiente resultado:

$$\mu_{y|x,z} = \beta_0 + \beta_1 Z_1 + \beta_2 Z_2 + \ldots + \beta_q Z_q + \beta_{k+1} X_{k+1}$$

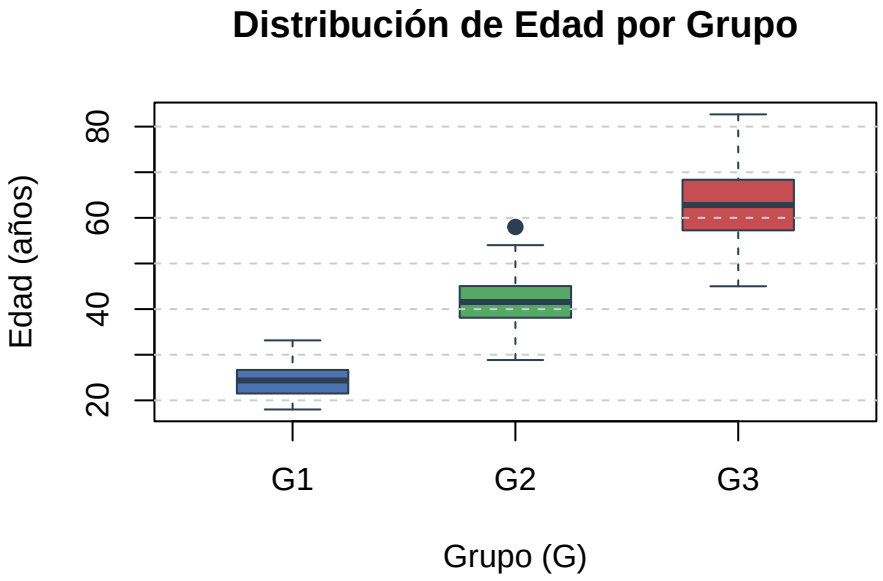
En este modelo la primera parte es no supervisado ya que no utilzamos la variable respuesta, para el último modelo obtenido si es supervisado ya que se utiliza con la variable respuesta.

**Ejemplo 2**

1. Se tienen $X_1, \ldots, X_p$ Grupos
2. Formo $k$ grupos
3. Se crea una variable $G$, la cual se crea a partir del aprendizaje no supervisado. Ya que se crea basado en las X's

| Observación $(i)$ | $X_1$ | $X_2$ | $\ldots$ | $X_p$ | $G = f(X_1, \ldots, X_p)$ |
|---|---|---|---|---|---|
| 1 | $x_{11}$ | $x_{12}$ | $\ldots$ | $x_{1p}$ | $g_1$ |
| 2 | $x_{21}$ | $x_{22}$ | $\ldots$ | $x_{2p}$ | $g_2$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ | $\vdots$ |
| $i$ | $x_{i1}$ | $x_{i2}$ | $\ldots$ | $x_{ip}$ | $g_i$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ | $\vdots$ |
| $n$ | $x_{n1}$ | $x_{n2}$ | $\ldots$ | $x_{np}$ | $g_n$ |

*Nota:* Si agregamos una variable edad, podríamos realizar un análisis de varianza, ya que dicha variable funcionaría como una variable respuesta.

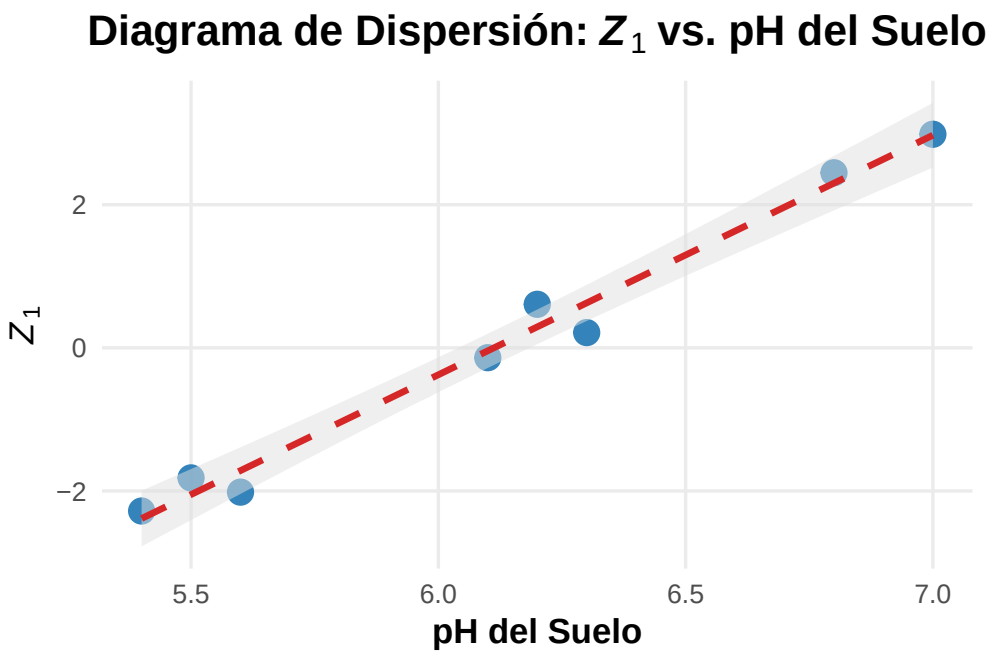



**Ejemplo 3**

| Observación ($i$) | Altura (cm) | N.° de Hojas | Grosor del Tallo (mm) | Peso del Racimo (kg) | pH del Suelo |
|---|---|---|---|---|---|
| 1 | 24.5 | 6 | 3.2 | 0.45 | 5.4 |
| 2 | 28.1 | 7 | 3.8 | 0.58 | 5.6 |
| 3 | 31.4 | 8 | 4.1 | 0.65 | 5.5 |
| 4 | 52.6 | 14 | 7.5 | 1.85 | 6.1 |
| 5 | 58.0 | 15 | 8.2 | 2.10 | 6.3 |
| 6 | 61.3 | 17 | 8.9 | 2.40 | 6.2 |
| 7 | 84.7 | 22 | 12.4 | 4.15 | 6.8 |
| 8 | 89.2 | 25 | 13.1 | 4.60 | 7.0 |

Note que la Altura (cm), N.° de Hojas, Grosor del Tallo (mm), Peso del Racimo (kg) están muy correlacionadas, mientras que el H del Suelo no tanto.

Con esas variables correlaciondas puede crear una variable $Z_1$




## 5 Proceso de Análisis de componentes principales (PCA)

$$\mu_{y|x_1,x_2,x_3,x_4} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4$$

Suponga que $X_2, X_3, X_4$ están muy correlacionadas, para ello se crea una variable $Z_1$.

$$\mu_{y|x_1,z_1} = \beta_0 + \beta_1 X_1 + \gamma_1 Z_1$$

Sin embargo, como pasamos de tres variables $(X_2, X_3, X_4)$ a una $(Z_1)$ perdemos cierta información. Es decir $Z_1$ explica un cierto porcentaje de $(X_2, X_3, X_4)$. Por ejemplo $Z_1$ explica un 80% de $(X_2, X_3, X_4)$.

Se puede dar el caso que $Z_1$ no explique suficiente.

Existe otro modelo que puede explicar más que el siguiente:

$$\mu_{y|x_1,x_3} = \beta_0 + \beta_1 X_1 + \beta_3 X_3$$

*Nota:* Es importante resaltar que los tres modelos anteriores no son lo mismo, cada uno tiene sus ventajas y desventajas.

El PCA sirve para eliminar la multicolinealidad, solo si nuestro objetivo es usarlo para una regresión. Otro uso que se le puede dar es para obtener gráficos más simples.

## 5.1 Maldición de la dimensionalidad

- Cuando se cuenta con demasidadas variables es difícil hacer un análisis gráfico para tener una idea inicial del comportamiento de los datos.
- Muchas variables son un obstáculo para aplicar otras técnicas multivariadas.

## 5.2 Requisitos para que PCA tenga sentido

- Si las variables originales casi no están correlacionadas, nada se gana al llevar a cabo un PCA.
- Cuando el determinante de una matriz es cero, se tiene que hay columnas linealmente dependientes. Esto se llama multicolinealidad.
- Si el determinante de la matriz de correlación (R) es cercano a cero, hay indicación de altas correlaciones entre las variables, de lo contrario, cuando el determinante es alto un PCA puede no tener sentido.

## 5.3 Ejemplo de aplicación

Se tienen los siguientes datos

| Observación ($i$) | $X_1$ | $X_2$ | $X_3$ | $X_4$ |
|---|---|---|---|---|
| 1 | 3 | 4 | 2 | 5 |
| 2 | 5 | 1 | 1 | 3 |
| 3 | 4 | 3 | 3 | 2 |
| 4 | 2 | 7 | 4 | 0 |
| $\bar{X} \rightarrow$ | 3.6 | 3.75 | 2.5 | 2.5 |
| $S \rightarrow$ | 1.24 | 2.5 | 1.24 | 2.05 |

Luego estándarizamos mediante la siguiente fórmula:

$$X_i = \frac{X_i - \bar{X}}{S_i}$$

```r
# 1. Definición de la matriz de datos X (4 observaciones x 4 variables)
X <- matrix(
  c(
    3, 4, 2, 5,
    5, 1, 1, 3,
    4, 3, 3, 2,
    2, 7, 4, 0
  ),
nrow = 4,byrow = TRUE,dimnames = list(paste0("ID_", 1:4), paste0("X", 1:4)))
# 2. Visualizar resultados
print("Matriz Original X:")
```

```
## [1] "Matriz Original X:"
```

```
print(X)
```

```
##      X1 X2 X3 X4
## ID_1  3  4  2  5
## ID_2  5  1  1  3
## ID_3  4  3  3  2
## ID_4  2  7  4  0
```

```
# 2. Estandarización con scale()
# scale() por defecto resta la media muestral (center = TRUE)
# y divide entre la desviación estándar muestral s con n-1 grados de libertad (scale = TRUE)
Z <- scale(X, center = TRUE, scale = TRUE)
print("Matriz Estandarizada Z (Z-scores):")
```

```
## [1] "Matriz Estandarizada Z (Z-scores):"
```

```
print(round(Z, 4))
```

```
##           X1   X2      X3      X4
## ID_1 -0.3873  0.1 -0.3873  1.2010
## ID_2  1.1619 -1.1 -1.1619  0.2402
## ID_3  0.3873 -0.3  0.3873 -0.2402
## ID_4 -1.1619  1.3  1.1619 -1.2010
## attr(,"scaled:center")
##   X1   X2   X3   X4
## 3.50 3.75 2.50 2.50
## attr(,"scaled:scale")
##       X1       X2       X3       X4
## 1.290994 2.500000 1.290994 2.081666
```

**Interpretación** El ID_4 en $X_1$ esta 1.16 desviaciones estándar debajo la media (recuerde que la desvación estándar de $X_1$ es 1.29). También en $X_2$ el ID_1 está 0.1 desviaciones estándar sobre la media, es decir está practicamente en la media.

*Nota:* El máximo de desviaciones estándar en una normal son 3 que son el 99.7% de los datos.

Luego obtenemos los coeficientes para $Z_1$ y hacemos la siguiente regresión:

$Z_1 = 0.2X_1^s + 0.4X_2^s + 0.01X_3^s + 0.5X_4^s$

*Nota:* En regresión estandarizabamos los coeficientes para poder compararlos en este caso, estadarizamos y la que tiene más peso en el resultado de $Z_1$ es $X_4$

Luego sustituimos con los valores que tenemos

$$\begin{aligned} Z_1^{(1)} &= 0.2(-0.39) + 0.4(0.10) + 0.01(-0.39) + 0.5(1.20) \\ &= -0.078 + 0.040 - 0.0039 + 0.600 = \mathbf{0.5581} \end{aligned}$$

$$\begin{aligned} Z_1^{(2)} &= 0.2(1.16) + 0.4(-1.10) + 0.01(-1.16) + 0.5(0.24) \\ &= 0.232 - 0.440 - 0.0116 + 0.120 = \mathbf{-0.0996} \end{aligned}$$

$$\begin{aligned} Z_1^{(3)} &= 0.2(0.39) + 0.4(-0.30) + 0.01(0.39) + 0.5(-0.24) \\ &= 0.078 - 0.120 + 0.0039 - 0.120 = \mathbf{-0.1581} \end{aligned}$$

$$\begin{aligned} Z_1^{(4)} &= 0.2(-1.16) + 0.4(1.30) + 0.01(1.16) + 0.5(-1.20) \\ &= -0.232 + 0.520 + 0.0116 - 0.600 = \mathbf{-0.3004} \end{aligned}$$

*Nota:* Si uno tiene $p$ variables originales, puede llegar a $p$ variables nuevas, la idea es siempre obtener menos que $p$

Más adelante se explicará la obtención de los $Z$'s mediante la descomposición esprectral.

Además los coeficentes de los $Z$'s, como está normalizados la suma de ellos es igual a 1.

**Sub-Ejemplo**

Se tiene el siguiente $Z$

$$Z_2 = 0.8X_1 + 0.8X_2 + 0.01x_3 + 0.01X_4$$

Como $X_3, X_4$ prácticamente no pesan, por lo que se interpretaría como un promedio entre $X_1$ y $X_2$

Para este otro caso

$$Z_3 = 0.8X_1 + 0.6X_2 + 0.5x_3 + 0.01X_4$$

En esta situación es una especie de ponderación donde el que tiene más peso es $X_1$

Caso inusual

$$Z_4 = 0.5X_1 - 0.5X_2 + 0.01x_3 + 0.01X_4$$

- Si $X_1 = X_2 = 0$ significa que está muy parecido
- Si $X_1 > X_2$

Resultado algebraico: La diferencia $(X_1 - X_2)$ es positiva, por lo que $Z_4 > 0$.

Interpretación: La unidad de estudio (persona, empresa o institución) tiene un predominio o sesgo a favor de la variable $X_1$ sobre $X_2$. Cuanto mayor sea el valor positivo de $Z_4$, más sobresale $X_1$ en relación con $X_2$.

- Si $X_1 < X_2$

Resultado algebraico: La diferencia $(X_1 - X_2)$ es negativa, por lo que $Z_4 < 0$.

Interpretación: La unidad de estudio presenta un predominio o concentración hacia $X_2$ sobre $X_1$. Cuanto más negativo sea $Z_4$, mayor es la ventaja o concentración relativa de $X_2$ respecto a $X_1$.

# 6 Comparación de análisis factorial con componentes principales

| **Factores comunes** | **Componentes principales** |
|---|---|
| $X_1 = \lambda_{11}f_1 + \lambda_{12}f_2 + \cdots + \lambda_{1m}f_m + \eta_1$ | $Z_1 = a_{11}X_1^s + a_{12}X_2^s + \cdots + a_{1q}X_q^s$ |
| $X_2 = \lambda_{21}f_1 + \lambda_{22}f_2 + \cdots + \lambda_{2m}f_m + \eta_2$ | $Z_2 = a_{21}X_1^s + a_{22}X_2^s + \cdots + a_{2q}X_q^s$ |
| $\vdots$ | $\vdots$ |
| $X_q = \lambda_{q1}f_1 + \lambda_{q2}f_2 + \cdots + \lambda_{qm}f_m + \eta_q$ | $Z_q = a_{q1}X_1^s + a_{q2}X_2^s + \cdots + a_{qq}X_q^s$ |

- La estructura del análisis de factores es similar a los componentes principales.
- En componentes principales las variables conocidas $X$ se combinan con los coeficientes a para producir nuevas variables $Z$ llamadas componentes principales. Una propiedad importante en el análisis de componentes principales es $Cov(Z_i, Z_j) = 0$
- En factores se combinan factores latentes desconocidos $f$ para formar las variables conocidas $X$. Esto es más complejo

### 6.1 Ejemplo de análisis factorial

Tenemos un estudio con $f_1$ que es la habilidad cuantitativa de una persona y $f_2$ es la habilidad de razonamiento verbal, para estos dos variables no tienen una forma directa de medirse, ya que $f_1$ se podría medir mediante un problema o un cuestionario, pero dentro de las preguntas del cuestionario, mientras se mide la habilidad cuantivativa, también se podría estar midiendo el razonamiento verbal, ya que la persona necesita entender la pregunta para resolver el problema.

Por ejemplo una persona obtuvo en una pregunta un valor de $f_1 = 0.5$ y $f_2 = 0.5$, asuma que $\lambda_{11} = 7$ y $\lambda_{12} = 7.5$

Esto nos lleva a que

$$7.25 = 0.5 * 7 + 0.5 * 7.5$$

En otra pregunta se tiene el resultado de otro puntaje, sin embargo lo que cambia es $f_1$ y $f_2$, en el caso de los $\lambda$'s no cambian y se pueden obtener por medio de **PCA**, esta es la confusión que muchas personas tienen con este tema.

### 6.2 Diferencias entre análisis factorial y PCA

| Factores comunes | Componentes principales |
|---|---|
| • El interés es la explicación de la **estructura** de covarianza o de correlación (o ambas) entre las variables medidas. | • El interés se centra en la explicación de la **variabilidad total** del conjunto de variables (Como una regresión). |
| • Puede haber **cambios sustanciales** en todos los factores si se cambia el número de factores. | • Si el número de componentes que se utilizan se incrementa, esos componentes **no cambian**. |
| • El cálculo de los puntajes es **complejo** y existen varios métodos. | • El cálculo de los puntajes es **sencillo** y se basa en una única descomposición espectral. |
| • Se obtienen resultados **equivalentes** si se usa la matriz de covarianza o la de correlaciones. | • **No hay relación** directa entre los componentes obtenidos de la matriz de covarianza y la de correlaciones. |
| • Si las variables están casi no correlacionadas este análisis no tiene sentido. | • Si las variables están casi no correlacionadas este análisis no tiene sentido. |
| • En tal caso los factores no tienen nada que explicar. | • En tal caso se obtienen componentes que son muy similares a las variables originales. |

*Nota:* Al agregar $Z$'s en componentes principales, las coeficientes para obtener los componentes no cambien, en cambio en análisis factorial al yo incluir más cosas que quiero medir ($X$'s), las $f$ cambian. Además, si las variancias específicas son pequeñas ambos análisis dan resultados similares.

## 7 Valores y Vectores propios

A continuaciión el siguiente tema se explica mediante un ejemplo.

Tenemos la siguiente matriz de varianzas y covarianzas:

$$\mathbf{S} = \begin{bmatrix} 8 & 10 & -3 \\ 10 & 6 & -4 \\ -3 & 4 & 2 \end{bmatrix}$$

Note que la $tr(S) = 8 + 6 + 2 = 16$, esta corresponde a la suma de las varianzas, lo que signica que es la **Varianza total**

Una vez teniendo esto podemos calcular la matriz de correlación por medio de la siguiente fórmula:

$$r_{ij} = \frac{s_{ij}}{\sqrt{s_{ii}s_{jj}}}$$

*Nota:* Recordar que en la diagonal la correlación es 1 ya que es la misma variable.

$$\mathbf{R} = \begin{bmatrix} 1 & 1.44 & -0.75 \\ 1.44 & 1 & -1.16 \\ -0.75 & 1.16 & 1 \end{bmatrix}$$

La $tr(R) = 3$, esta representa el número de variables. Además la matriz de correlaciones es la estandarización de una matriz de varianzas y covarianzas en este caso, ya que la varianza de cada una de las variables es 1. Esto nos lleva a la siguiente equivalencia:

$$Cov(X_i^s, X_j^s) = r_{ij}$$

Así que la $tr(R) = 3$ representa que la varianza total de las variables estándarizadas.

## 7.1 ¿Cuándo centar y cuándo estandarizar?

Cuando se tienen variables con las mismas escalas solo se necesitan centrar, pero cuando son muy diferentes es altamente recomendado estandarizarlos.

## 7.2 Valores y vectores propios en R

### Valores propios

```
valores = c(8, 10, -3, 10,  6, 4,-3,  4,  2 )
S = matrix(
  valores,
  nrow = 3,
  ncol = 3,
  byrow = TRUE
)

v = eigen(S)
round(v$values,2)
```

```
## [1] 17.07  5.04 -6.11
```

Esto nos dice que $\lambda_1 = 17.07, \lambda_2 = 5.04, \lambda_3 = -6.11$

Note que la suma de estos valores n¿da como resultado la varianza total original

```
sum(v$values)
```

```
## [1] 16
```

### Vectores propios

Estos deben complir con la propiedad de que:

$$A \times a = \lambda \times a$$

*A continuación se le representa el proceso en r, para obtenerlos*

Tenemos la matriz asociada a los vectores propios

```
v$vectors
```

```
##            [,1]       [,2]       [,3]
## [1,] 0.73550267 -0.3755265  0.5639288
## [2,] 0.67670738  0.3663738 -0.6386215
## [3,] 0.03321052  0.8513226  0.5235904
```

Los extraemos de la misma matriz

```
e1 = v$vectors[, 1]  # Asociado a lambda_1 = 17.06
e2 = v$vectors[, 2]  # Asociado a lambda_2 = 5.05
e3 = v$vectors[, 3]  # Asociado a lambda_3 = -6.11
```

Verificamos con la condición de modo que `S %*% e_1 == lambda_1 * e_1`

```
t(round(S %*% e1, 4))
```

```
##         [,1]    [,2]   [,3]
## [1,] 12.5515 11.5481 0.5667
```

```
round(v$values[1] * e1, 4)
```

```
## [1] 12.5515 11.5481  0.5667
```

Observe que la condición se cumple.

También otra propiedad que se tiene que resaltar es la ornormalidad de los mismos.

Verificación sobre si la norma es 1

```
# Norma 1: ||e_1|| = 1
sqrt(sum(e1^2))
```

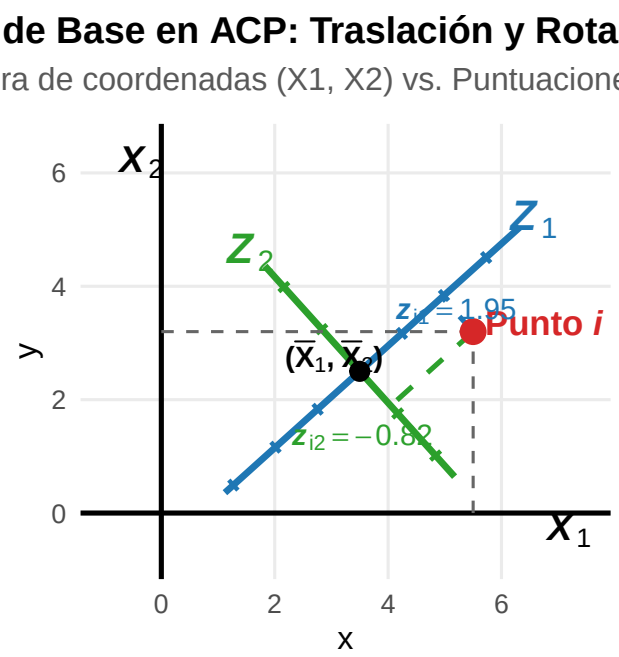
```
## [1] 1
```

Verificación de ortogonalidad

```
# Ortogonales entre sí: e_1 %*% e_2 = 0
round(t(e1) %*% e2, 4)
```

```
##      [,1]
## [1,]    0
```

## 7.3 Construcción de Z's




Observando la figura anterior tenemos que:

$X_1 = 5.5 \rightarrow Z_1 = 1.95$ y $X_2 = 3.2 \rightarrow Z_2 = -0.82$. Con lo siguiente $r_{x_1,x_2} = 0.45 \rightarrow r_{z_1,z_2} = 0$

¿Cómo se construyen los $Z$'s de la figura anterior?

- Si se utliza la matriz de correlaciones, se deben estandarizar las variables, esto se hace cuando las escalas de las variables son similares. $(X_1^s, X_2^s \rightarrow S)$
- Si se utiliza la matriz de covarianzas y varianzas, se debe centrar solamente, sin embargo esto se debe hacer solo cuando las escalas sean muy similares. $(X_1^c, X_2^c \rightarrow R)$

**Proceso de obtención de los Z's**

Sabemos que en el anterior ejercicio los vectores propios son:

```
## El primer vector propio es 0.74 0.67
## El segundo vector propio es -0.67 0.74
```

Tenemos que las medias son:

```
## Media de X1 es 3.5
## Media de X2 es 2.5
```

Hacemos las operaciones para llegar al resultado

$$\begin{aligned} Z_1 &= 0.74\, X_1^c + 0.67\, X_2^c \\ &= 0.74\,(5.5 - 3.5) + 0.67\,(3.2 - 2.5) \\ &= 0.74\,(2.0) + 0.67\,(0.7) \\ &= 1.48 + 0.469 \\ &= 1.95 \end{aligned}$$

$$
\begin{aligned}
Z_2 &= -0.67\, X_1^c + 0.74\, X_2^c \\
&= -0.67\,(5.5 - 3.5) + 0.74\,(3.2 - 2.5) \\
&= -0.67\,(2.0) + 0.74\,(0.7) \\
&= -1.34 + 0.518 \\
&= -0.82
\end{aligned}
$$

Haciendolo en una base de datos grande lo que se está haciendo es lo siguiente:

$$
\mathbf{X}^c = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & x_{n3} \end{bmatrix} - \begin{bmatrix} \bar{x}_1 & \bar{x}_2 & \bar{x}_3 \\ \bar{x}_1 & \bar{x}_2 & \bar{x}_3 \\ \vdots & \vdots & \vdots \\ \bar{x}_1 & \bar{x}_2 & \bar{x}_3 \end{bmatrix}
$$

Luego se multiplica por el vector propio:

$$
\mathbf{X}^c = \begin{bmatrix} x_{11}^c & x_{12}^c & x_{13}^c \\ x_{21}^c & x_{22}^c & x_{23}^c \\ \vdots & \vdots & \vdots \\ x_{n1}^c & x_{n2}^c & x_{n3}^c \end{bmatrix} \cdot \begin{bmatrix} a_{11} \\ a_{22} \\ a_{33} \end{bmatrix} = \begin{bmatrix} Z_{1(1)} \\ Z_{2(1)} \\ \vdots \\ Z_{n(1)} \end{bmatrix}
$$

Con esto llegamos obtener el vector con los valores de la primera variable nueva, para todos los individuos de la base de datos o lo que es lo mismo que decir que el vector del primer componente principal.

*Nota:* Si se calculan las varianzas de los $Z$'s da como resultado los $\lambda$'s encontrados para los vectores propios, por lo que la variabilidad total es igual si se mantiene el mismo número de variables originales.

## 7.4 Estudio de intervalos en los que se encuentran los datos generados por un componente

Tenemos los siguientes datos

**Valores propios**

| PC1 | PC2 | PC3 | PC4 |
|---|---|---|---|
| 2.73 | 0.76 | 0.38 | 0.13 |

**Vectores propios**

| Variable | PC1 | PC2 | PC3 | PC4 |
|---|---|---|---|---|
| vallas | -0.57 | -0.07 | -0.18 | -0.80 |
| saltoalto | -0.38 | 0.89 | -0.14 | 0.22 |
| tiro | -0.51 | -0.14 | 0.82 | 0.20 |
| car200 | -0.51 | -0.43 | -0.52 | 0.53 |

Al obtener los puntajes del primer componente, ¿será posible que uno de ellos sea 8.1?

Debemos aplicar la siguiente propiedad:

$$
Var(Z_j) = \lambda_j \implies sd(Z_j) = \sqrt{\lambda_j}
$$

```r
vp1 = 2.73
desv1 = sqrt(vp1)
desv1
```

```
## [1] 1.652271
```

```r
2*desv1
```

```
## [1] 3.304542
```

```r
3*desv1
```

```
## [1] 4.956813
```

No es posible que uno de los valores del primer componente sea 8.1. Si consideramos que la desviación estándar es de 1.65, y que la media es de cero (por propiedades de estandarización, se espera que los valores estén en el rango entre -2 y 2 desviaciones estándar, es decir entre -3.3 y 3.3

### 7.5 Correlación entre Componentes y variables

uando el Análisis de Componentes Principales se realiza sobre variables estandarizadas (usando la matriz de correlaciones $\mathbf{R}$), la correlación entre la $i$-ésima variable original estandarizada $Z_i$ y el $j$-ésimo componente principal $Y_j$ (llamada comúnmente factor loading o carga factorial) viene dada por la propiedad:

$$\text{Corr}(Z_i, Y_j) = a_{ij}\sqrt{\lambda_j}$$

donde:

- $a_{ij}$ es el coeficiente (peso) de la variable
- $i$ en el vector propio del componente $j$.
- $\lambda_j$ es el valor propio asociado al componente $j$ (que corresponde a su varianza).

Dado esta propiedad, basado en el anterior ejemplo:

¿Cómo espera que sea la correlación entre el segundo componente principal y la variable C10? Justifique su respuesta

```r
coef_saltoalto = 0.89 #el coeficiente de la variable en el segundo componente
lambda_2 = 0.76 # la varianza del segundo componente

coef_saltoalto*sqrt(lambda_2)
```

```
## [1] 0.775884
```

## 8 Aplicación de PCA ejemplo de HEPTATLON

### 8.1 Proceso de cambios en las variables

Para cambiar el sistema de puntuación realizamos la siguiente transformación de tal manera que:

| Observación ($i$) | $\rightarrow$ | Observación Nueva |
|---|---|---|
| 3 | | 7 |
| 5 | | 5 |
| 8 | | 2 |
| 10 | | 0 |

La fórmula utlizada es:

$$x_{new=}max - x_i$$

```
load("Heptatlon.Rdata")
base$vallas=with(base,max(vallas)-vallas)
base$car200=with(base,max(car200)-car200)
base$car800=with(base,max(car800)-car800)
```

## 8.2 Matriz de correlaciones

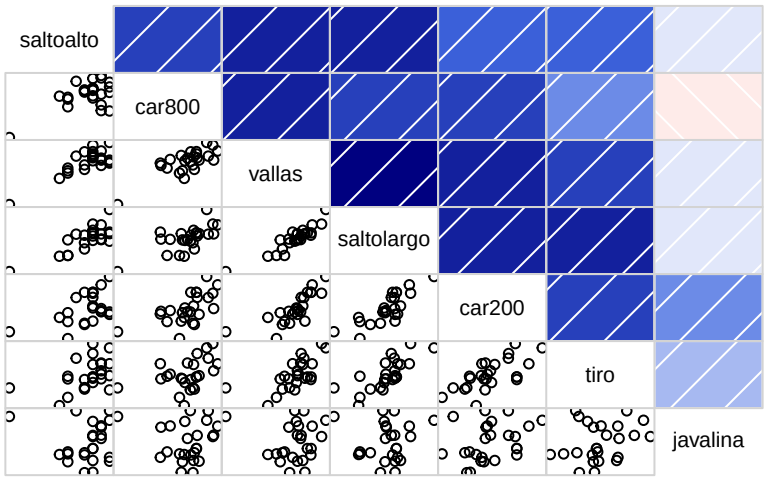
### 8.2.1 Cálculo en r

```
round(cor(base[,-8]), 2)
```

```
##             vallas saltoalto tiro car200 saltolargo javalina car800
## vallas        1.00      0.81 0.65   0.77       0.91     0.01   0.78
## saltoalto     0.81      1.00 0.44   0.49       0.78     0.00   0.59
## tiro          0.65      0.44 1.00   0.68       0.74     0.27   0.42
## car200        0.77      0.49 0.68   1.00       0.82     0.33   0.62
## saltolargo    0.91      0.78 0.74   0.82       1.00     0.07   0.70
## javalina      0.01      0.00 0.27   0.33       0.07     1.00  -0.02
## car800        0.78      0.59 0.42   0.62       0.70    -0.02   1.00
```

### 8.2.2 Gráfico de correlación librería corrgram de colores

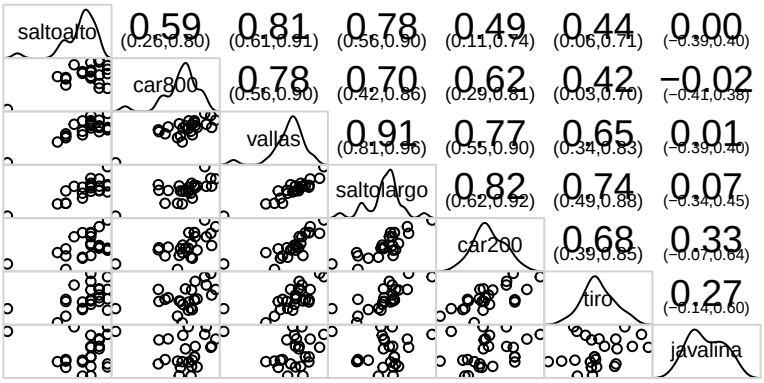
```
library(corrgram)
corrgram(base[,-8], lower.panel=panel.pts,order=T)  # order = T nos orderna las correlaciones
```




Los colores azules significan una correlación positiva, mientras que los rojos significa una correlación negatiiva. Además , en los gráficos de puntos, en los ejes están los valores de cada variables según corresponda.

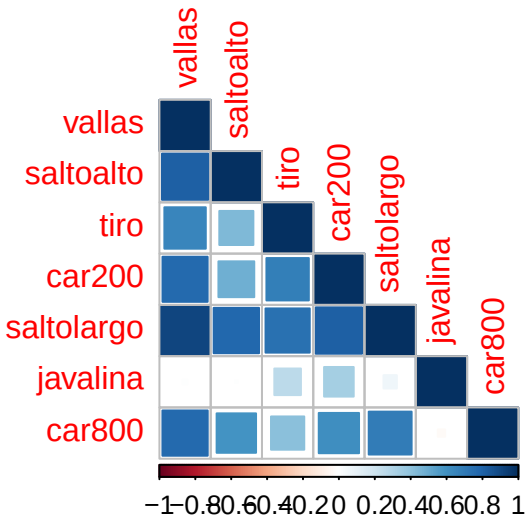
### 8.2.3 Gráfico de correlación librería corrgram de números (correlaciones)

```
corrgram(base[,-8], lower.panel=panel.pts, upper.panel=panel.conf,
         diag.panel=panel.density,order=T)
```




### 8.2.4 Gráfico del paquete corrplot

```
library(corrplot)
corrplot(cor(base[,-8]), method = "square", type = "lower")
```




```
# El parámetro lower ordena el correlograma de mannera decendente
```

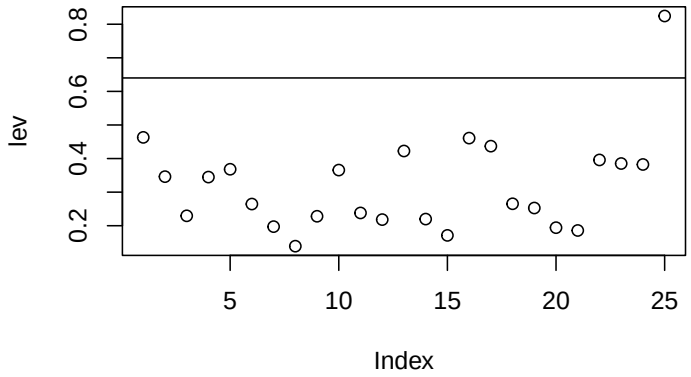
## 8.3 Análisis de valores extremos mendiante leverage

Recuerde que este proceso se realiza mediante el cálculo de la matriz:

$$H = X(X^T X)^{-1} X^T$$

*A continuación se le presenta el código en r, para los leverage*

```
lev=hat(base[,-8])
plot(lev)
lim=2*mean(lev)
abline(h=lim)
```




### 8.3.1 Formas de averiguar y eliminar el valor extremo

**Formas de averiguar**

```
which.min(base$saltoalto) # Es el punto menor que se ve en los gráficos de correlación más arriba
```

```
## [1] 25
```

```
which.max(lev) # El valor más alto que se sale de la linea en los leverage
```

```
## [1] 25
```

**Formas de eliminar**

```
base1 = subset(base, lev<lim)
```

```
base1 = base[-25,]
```

*Nota:* Recordar que en este caso se quitó el valor extremo por términos del ejericicio y nunca se puede quitar sin una justificación en concreto.

## 8.4 Obtención del determinante de la matriz R

```
base2=base1[,-8] # Es una base sin la respuesta (solo con las X's)
R=cor(base2)
det(R)
```

```
## [1] 0.003511559
```

Conforme las correlaciones, aumenta el determinante tiende a cero.

$$\text{Correlación alta} \implies \text{Determinante casi cero}$$

## 8.5 Hipótesis de matriz de correlaciones

Existe una prueba de hipótesis que verifica si la matriz de correlaciones es igual a la identidad, ya que si es similar a la identidad, indicaría que las correlaciones son prácticamente cero.

*Nota:* Para hacer el análisis de PCA **mínimo**, se debe rechazar esta hipótesis.

$$H_0 : R = I$$

```
library(psych)
cortest.bartlett(R,n=nrow(base2)) # Proporcionadole a la función la matriz
```

```
## $chisq
## [1] 112.092
##
## $p.value
## [1] 1.973466e-14
##
## $df
## [1] 21
```

```
cortest.bartlett(base2) # En esta opción, la base de datos no puede contener la respuesta
```

```
## $chisq
## [1] 112.092
##
## $p.value
## [1] 1.973466e-14
##
## $df
## [1] 21
```

## 8.6 KMO

En ocasiones la prueba de correlaciones falla debido a la gran cantidad de datos, ya que debido a la gran potencia que tienen, ocasiona que la prueba se rechaze siempre.

Una solución ante este problema es usar esta prueba, que tiene como condición que solo si $n/q < 5$, donde q es el número de variables.

*A continuación se le presentan los criterios del KMO*

| Rango KMO | Interpretación / Calidad |
|---|---|
| $[0.9, 1.0]$ | Muy bueno (Excelente) |
| $[0.8, 0.9]$ | Meritorio |
| $[0.7, 0.8]$ | Mediano (Aceptable) |
| $[0.6, 0.7]$ | Mediocre (Regular) |
| $[0.5, 0.6]$ | Bajo |
| $[0.0, 0.5]$ | Inaceptable (Desaconsejable) |

```
KMO(R) #library(psych)
```

```
## Kaiser-Meyer-Olkin factor adequacy
## Call: KMO(r = R)
## Overall MSA =  0.73
## MSA for each item =
##     vallas  saltoalto       tiro     car200 saltolargo   javalina     car800
##       0.87       0.55       0.82       0.72       0.72       0.47       0.84
```

En este caso la Correlación es aceptable

## 8.7 Análisis con la matriz de covarianzas

Como se mencionó anteriormente, este análisis se recomienda hacerlo cuando las varianzas de las variables son bastantes similares, ya que en este caso las variables no están estandarizas, tal y como sucede con el análisis de la matriz de correlación.

### 8.7.1 Comparación de varianzas de la variables

```
v=apply(base2, 2,var) # Note que el 2 es porque es la varianza para cada columna
round(v,3)
```

```
##     vallas  saltoalto       tiro     car200 saltolargo   javalina     car800
##      0.265      0.003      2.241      0.878      0.161     12.032     37.789
```

*Nota:* Con ver este resultado podemos concluir que no tiene sentido hacer un análisis por la matriz de covarianzas debido a la gran diferencia entre varianzas.

Este cálculo lo puede hacer también de la siguiente forma:

```
round(diag(var(base2)),3)
```

```
##     vallas  saltoalto       tiro     car200 saltolargo   javalina     car800
##      0.265      0.003      2.241      0.878      0.161     12.032     37.789
```

La variabilidad total de las variables originales la obtenemos, sumando todas las varianzas

```
sum(v)
```

```
## [1] 53.36842
```

### 8.7.2 Descomposición espectral de matriz de covarianzas

**Obtenemos la matriz de covarianzas**

```
S=var(base2)
round(S,2)
```

```
##            vallas saltoalto tiro car200 saltolargo javalina car800
## vallas       0.26      0.02 0.59   0.40       0.18     0.59   1.77
## saltoalto    0.02      0.00 0.04   0.02       0.01     0.06   0.05
## tiro         0.59      0.04 2.24   0.94       0.47     1.78   3.76
## car200       0.40      0.02 0.94   0.88       0.31     1.53   3.30
## saltolargo   0.18      0.01 0.47   0.31       0.16     0.40   1.29
## javalina     0.59      0.06 1.78   1.53       0.40    12.03   5.46
## car800       1.77      0.05 3.76   3.30       1.29     5.46  37.79
```

$$S = A\Lambda A' = A\Lambda^{\frac{1}{2}}\Lambda^{\frac{1}{2}}A'$$

donde:

$$\mathbf{\Lambda} = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

Por medio de `eigen()` hacemos la descomposición

```
espec=eigen(S)
```

*A continuación se le presenta código en r, de como obtener los valores y vectores propios*

```
lambda=espec$val
round(lambda,2)
```

```
## [1] 39.82 11.11  2.00  0.35  0.06  0.02  0.00
```

*Nota:* Observe que la suma de los $\lambda$' es lo mismo que la varianza total

```
sum(lambda) # Da lo mismmo que la varianza total (summa de las varianzas de cada predictor)
```

```
## [1] 53.36842
```

Esto nos lleva a escribir esta propiedad

$$\sum_{i=1}^{q} S_i^2 = \sum_{i=1}^{q} \lambda_j$$

```
A=espec$vec
round(A,2)
```

```
##       [,1]  [,2]  [,3]  [,4]  [,5]  [,6]  [,7]
## [1,] -0.05  0.03 -0.22 -0.26  0.77 -0.54  0.04
## [2,]  0.00  0.01 -0.01 -0.01  0.11  0.11 -0.99
## [3,] -0.11  0.11 -0.88  0.42 -0.14 -0.03 -0.01
## [4,] -0.09  0.08 -0.31 -0.85 -0.40 -0.03 -0.04
## [5,] -0.04  0.02 -0.18 -0.18  0.46  0.84  0.15
## [6,] -0.20  0.96  0.16  0.05  0.02  0.01  0.01
## [7,] -0.97 -0.23  0.11  0.05 -0.01  0.00  0.00
```

```
L = diag(lambda)
L
```

```
##          [,1]     [,2]     [,3]      [,4]       [,5]      [,6]         [,7]
## [1,] 39.82137  0.00000 0.000000 0.0000000 0.00000000 0.0000000 0.000000000
## [2,]  0.00000 11.11262 0.000000 0.0000000 0.00000000 0.0000000 0.000000000
## [3,]  0.00000  0.00000 2.003319 0.0000000 0.00000000 0.0000000 0.000000000
## [4,]  0.00000  0.00000 0.000000 0.3483982 0.00000000 0.0000000 0.000000000
## [5,]  0.00000  0.00000 0.000000 0.0000000 0.06044327 0.0000000 0.000000000
## [6,]  0.00000  0.00000 0.000000 0.0000000 0.00000000 0.0213549 0.000000000
## [7,]  0.00000  0.00000 0.000000 0.0000000 0.00000000 0.0000000 0.000921165
```

**Aplicamos la fórmula que esta arriba para obtener la matriz de covarianzas**

```
S1 = round(A%*%L%*%t(A),2)
S1
```

```
##      [,1] [,2] [,3] [,4] [,5]  [,6]  [,7]
## [1,] 0.26 0.02 0.59 0.40 0.18  0.59  1.77
```

```
## [2,] 0.02 0.00 0.04 0.02 0.01  0.06  0.05
## [3,] 0.59 0.04 2.24 0.94 0.47  1.78  3.76
## [4,] 0.40 0.02 0.94 0.88 0.31  1.53  3.30
## [5,] 0.18 0.01 0.47 0.31 0.16  0.40  1.29
## [6,] 0.59 0.06 1.78 1.53 0.40 12.03  5.46
## [7,] 1.77 0.05 3.76 3.30 1.29  5.46 37.79
```

## 8.8 Obtención de los componentes principales $Z$'s

```
basec=scale(base2,scale=F) # Datos centrados
Z1=basec%*%A[,1]
head(Z1,5) # Z1 de las 25 competidoras
```

```
##                           [,1]
## Joyner-Kersee (USA)  -7.647588
## John (GDR)           -9.244014
## Behmer (GDR)        -11.312888
## Sablovskaite (URS)   -3.185120
## Choubenkova (URS)    -8.287030
```

### 8.8.1 Cálculo de cuánto porcentaje explica el componente

$$\frac{Var(Z_1)}{Var_{total}} * 100$$

```
var(Z1)/sum(diag(var(base2)))*100
```

```
##          [,1]
## [1,] 74.61597
```

Otra forma de obtenerlo

```
lambda[1]/sum(lambda)*100
```

```
## [1] 74.61597
```

### 8.8.2 ¿Es normal que tenga media cero?

Es completamente normal, ya que cuando se usa la matriz de covarianzas al momento de centrar las variables, la media es prácticamente cero

```
mean(Z1)
```

```
## [1] 1.741662e-15
```

### 8.8.3 Forma automática con `prcomp()`

```
pca1=prcomp(base2) #Recordar que base2 no contiene la respuesta
summary(pca1)
```

```
## Importance of components:
##                           PC1    PC2     PC3     PC4     PC5    PC6     PC7
## Standard deviation     6.3104 3.3336 1.41539 0.59025 0.24585 0.1461 0.03035
## Proportion of Variance 0.7462 0.2082 0.03754 0.00653 0.00113 0.0004 0.00002
## Cumulative Proportion  0.7462 0.9544 0.99192 0.99845 0.99958 1.0000 1.00000
```

```
names(pca1)
```

```
## [1] "sdev"     "rotation" "center"   "scale"    "x"
```

`sdev`: Son las desviaciones estándar para obetner hago lo siguiente:

```
(round(pca1$sdev,2))^2
```

```
## [1] 39.8161 11.0889  2.0164  0.3481  0.0625  0.0225  0.0009
```

`rotation`: Es la matriz de los vectores propios

```
round(pca1$rotation,2) # Los vectores propios
```

```
##              PC1   PC2   PC3   PC4   PC5   PC6   PC7
## vallas     -0.05 -0.03  0.22  0.26 -0.77 -0.54 -0.04
## saltoalto   0.00 -0.01  0.01  0.01 -0.11  0.11  0.99
## tiro       -0.11 -0.11  0.88 -0.42  0.14 -0.03  0.01
## car200     -0.09 -0.08  0.31  0.85  0.40 -0.03  0.04
## saltolargo -0.04 -0.02  0.18  0.18 -0.46  0.84 -0.15
## javalina   -0.20 -0.96 -0.16 -0.05 -0.02  0.01 -0.01
## car800     -0.97  0.23 -0.11 -0.05  0.01  0.00  0.00
```

x: Es lo que contiene los $Z$'s

```
head(round(pca1$x,2),5)
```

```
##                        PC1   PC2   PC3   PC4   PC5   PC6   PC7
## Joyner-Kersee (USA)  -7.65 -3.28  1.93  0.61 -0.16  0.23 -0.04
## John (GDR)           -9.24  0.30  2.07 -0.61 -0.07 -0.15 -0.02
## Behmer (GDR)        -11.31 -0.99 -0.18  0.43  0.10  0.07  0.02
## Sablovskaite (URS)   -3.19 -1.13  1.51 -0.44  0.43 -0.10  0.04
## Choubenkova (URS)    -8.29 -4.61 -0.13 -0.63  0.18 -0.05 -0.06
```

*Note que da todos los z's de todos los individuos*

```
summary(pca1)
```

```
## Importance of components:
##                           PC1    PC2     PC3     PC4     PC5    PC6     PC7
## Standard deviation     6.3104 3.3336 1.41539 0.59025 0.24585 0.1461 0.03035
## Proportion of Variance 0.7462 0.2082 0.03754 0.00653 0.00113 0.0004 0.00002
## Cumulative Proportion  0.7462 0.9544 0.99192 0.99845 0.99958 1.0000 1.00000
```

Esto nos resume todo lo del análisis y cuánto aporta cada componente

# 9 Análisis con matriz de correlaciones

## 9.1 Estadarizando las variables prrimero

```
v1=scale(base2$vallas)
```

**Observe que la media es 0 y la varianza es 1.**

```
round(mean(v1),2)
```

```
## [1] 0
```

```
var(v1)
```

```
##      [,1]
## [1,]    1
```

Estandarice todas las variables. Para hacerlo de una sola vez, puede usar: `scale(base2)`.

```
base3=scale(base2)
```

Verificación de que todas las medias son 0 y las variancias son 1.

```
round(apply(base3,2,mean),10)
```

```
##     vallas  saltoalto       tiro     car200 saltolargo   javalina     car800
##          0          0          0          0          0          0          0
```

```
apply(base3,2,var)
```

```
##     vallas  saltoalto       tiro     car200 saltolargo   javalina     car800
##          1          1          1          1          1          1          1
```

## 9.2 Descomposición espectral de la matriz de correlaciones y obtención de los valores propios.

```
espec1=eigen(R)
lambda1=espec1$val
round(lambda1,2)
```

```
## [1] 4.32 0.90 0.83 0.47 0.30 0.11 0.07
```

```
S = var(base3)
S1 = var(base2) # esro no es lo mismo que S, no tiene sentido porque las variables no están estandariza
R = cor(base2)
```

Note que `S` y `R` son iguales

```
round(S,2) == round(R,2)
```

```
##            vallas saltoalto tiro car200 saltolargo javalina car800
## vallas       TRUE      TRUE TRUE   TRUE       TRUE     TRUE   TRUE
## saltoalto    TRUE      TRUE TRUE   TRUE       TRUE     TRUE   TRUE
## tiro         TRUE      TRUE TRUE   TRUE       TRUE     TRUE   TRUE
## car200       TRUE      TRUE TRUE   TRUE       TRUE     TRUE   TRUE
## saltolargo   TRUE      TRUE TRUE   TRUE       TRUE     TRUE   TRUE
## javalina     TRUE      TRUE TRUE   TRUE       TRUE     TRUE   TRUE
## car800       TRUE      TRUE TRUE   TRUE       TRUE     TRUE   TRUE
```

## 9.3 Porcentaje de explicación de un componente principal

Para averiguar ese porcentaje de explicación utlice la siguiente fórmula:

$$\frac{Var(Z_j)}{\sum_{j=1}^{q} Var(Z_j)}$$

donde $q$ es el número total de variables

*A continuación se le detalla el proceso en r*

Obtenemos el primer valor propio

```
lambda1[1]
```

```
## [1] 4.323642
```

Luego obtenemos el número total de variables o sumamos los valores propios.

```
sum(lambda1)
```

```
## [1] 7
```

```
round(lambda1[1]/sum(lambda1),2)
```

```
## [1] 0.62
```

Esto quiere decir que $Z_1$ explica un 62% de la variabilidad total, cuando las variables están estadarizadas.

## 9.4 Forma automática

```
pca1=prcomp(base2,scale=T)
lambda2=pca1$sdev**2
round(lambda2,2)
```

```
## [1] 4.32 0.90 0.83 0.47 0.30 0.11 0.07
```

Note que con `summary()`, podemos obtener directamente cuánto porcentaje explica cada componente.

```
summary(pca1)
```

```
## Importance of components:
##                           PC1    PC2    PC3     PC4     PC5     PC6     PC7
## Standard deviation     2.0793 0.9482 0.9109 0.68320 0.54619 0.33745 0.26204
## Proportion of Variance 0.6177 0.1284 0.1185 0.06668 0.04262 0.01627 0.00981
## Cumulative Proportion  0.6177 0.7461 0.8646 0.93131 0.97392 0.99019 1.00000
```

Obtención del primer vector propio

```
a1=pca1$rot[,1]
round(a1,2)
```

```
##     vallas  saltoalto       tiro     car200 saltolargo   javalina     car800
##      -0.45      -0.31      -0.40      -0.43      -0.45      -0.24      -0.30
```

Observe que la variables con más peso en este caso son: `vallas`, `saltolargo` y `saltolargo`. Además todos son negativos, esto es positiva ya que en terminos de interpretación estos se van a estabilizar.

**Es importante fijarse en el signo de los coeficientes que pesen más. Si hay uno que pese muy poco comparado con los demás, pero tiene signo diferente lo que se puede hacer es ignorarlo ya que no va a afectar tanto**

## 9.5 Obtención del primer componente manualmente

```
Z1=base3%*%a1
Z1a=pca1$x[,1]
cbind(Z1,Z1a)
```

```
##                                       Z1a
## Joyner-Kersee (USA) -4.757530189 -4.757530189
## John (GDR)          -3.147943402 -3.147943402
## Behmer (GDR)        -2.926184760 -2.926184760
## Sablovskaite (URS)  -1.288135516 -1.288135516
## Choubenkova (URS)   -1.503450994 -1.503450994
## Schulz (GDR)        -0.958467101 -0.958467101
## Fleming (AUS)       -0.953445060 -0.953445060
## Greiner (USA)       -0.633239267 -0.633239267
```

```
## Lajbnerova (CZE)    -0.381571974 -0.381571974
## Bouraga (URS)       -0.522322004 -0.522322004
## Wijnsma (HOL)       -0.217701500 -0.217701500
## Dimitrova (BUL)     -1.075984276 -1.075984276
## Scheider (SWI)       0.003014986  0.003014986
## Braun (FRG)          0.109183759  0.109183759
## Ruotsalainen (FIN)   0.208868056  0.208868056
## Yuping (CHN)         0.232507119  0.232507119
## Hagger (GB)          0.659520046  0.659520046
## Brown (USA)          0.756854602  0.756854602
## Mulliner (GB)        1.880932819  1.880932819
## Hautenauve (BEL)     1.828170404  1.828170404
## Kytola (FIN)         2.118203163  2.118203163
## Geremias (BRA)       2.770706272  2.770706272
## Hui-Ing (TAI)        3.901166920  3.901166920
## Jeong-Mi (KOR)       3.896847898  3.896847898
```

En contexto de este problema mientras más pequeño sea el número mejor es la puntuación, esto nos lleva que la atleta `Jeong-Mi (KOR)`, es la que peor puntuación tiene ya que tiene un valor de $Z_1$ alto. Otro aspecto importante a notar es que `Scheider (SWI)` se mantiene muy cerca del promedio, esto debido a su valor en el componente es casi cero.

### 9.6 Comparación de Varianzas y lambdas

```
var(Z1a)
```

```
## [1] 4.323642
```

```
lambda1[1]
```

```
## [1] 4.323642
```

Observe que son iguales, **recuerde que esto siempre va a suceder**

### 9.7 Cambio a los signos de los coeficientes de $Z_1$

```
Z1b=-Z1a
var(Z1b)
```

```
## [1] 4.323642
```

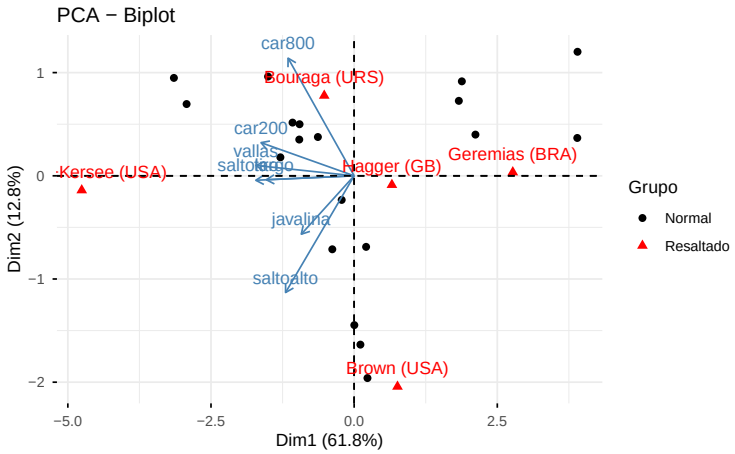
*Nota:* Recuerde que la varianza es la misma ya que las constantes salen al cuadrado, por lo que el menos que le incluímos se le anula.

## 10 Biplot

Este es el código para hacer el gráfico normal

```
biplot(pca1, cex=0.6)
```

*El gráfico que se le presenta a continuación fue arreglado para términos de presentación y explicación*




Del gráfico podemos notar que la paricipante `Joyner-Kersee (USA)` tienen a tener puntajes altos por ello se ubica en esa posición. Por otro lado esta la participante `Geremias (BRA)`, esta posicionada al otro lado completamente de la participamos anteriormente, dicha posición es es resultado de que a la participante `Geremias (BRA)` no le fue tan bien en las tres flechas que van en esa dirección (`vallas`, `saltolargo` y `saltolargo`).

Se puede observar que todas las flechas van hacia la izquierda debido a que su signo es negativo. Además, se puede ver que las que pesan más en el primer componente son aquellas que tienen una flecha más larga, por ejemplo en el gráfico anterior `javalina` es la que tiene una menor longitud en la flecha debido a que el peso de este es menor que el de las demás variables. Por otro lado, las que más pesan como habíamos visto anteriormente son `vallas`, `saltolargo` y `saltolargo`, estas mismas están agrupadas en este gráfico.

Para el segundo componente primero debemos observar el siguiente vector propio:

```
a2=pca1$rot[,2]
round(a2,2)
```

```
##     vallas  saltoalto       tiro     car200 saltolargo   javalina     car800
##       0.06      -0.65      -0.02       0.19      -0.02      -0.33       0.66
```

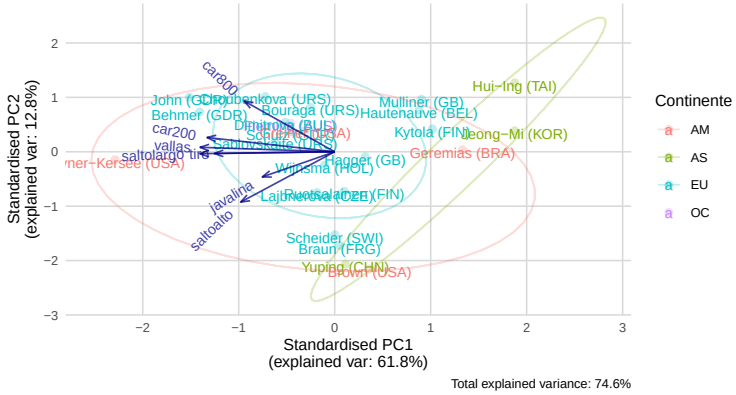
Note que para este caso las que tiene más peso son las variables `saltoalto` y `car800`, por esta razón estos son los que tiene una flecha de mayor tamaño con respecto al eje vertical. Además note que el signo de `car800` es positivo, por eso es que la flecha tiene una dirección hacia arriba.

Un aspecto importante a resaltar de la participante `Brown (USA)`, es que le fue mejor en `saltoalto` que en `car800`, mientras que en `vallas`, `saltolargo` y `saltolargo` está en el promedio, en el otro extremo del gráfico la participante `Bouraga (URS)`, le fue de manera intermedia (o se encuentra en el promedio) en las otras variables, pero le fue mucho mejor en `car800` en comparación con `saltoalto`.

## 10.1 Biplot categórico

```
library(AMR)
library(dplyr)
data=data.frame(c("AM","EU","EU","EU","EU","EU","OC","AM","EU",
                  "EU","EU","EU","EU","EU","EU","AS","EU","AM",
                  "EU","EU","EU","AM","AS","AS"),row.names(base2),base2)
names(data)=c("Continente","Nombre",names(base2))
pca2=data %>%
  pca(vallas,saltoalto,tiro,car200,saltolargo,javalina,car800)
```

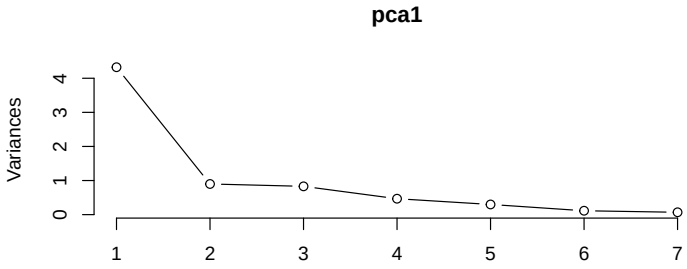
```
ggplot_pca(pca2)
```




# 11 Gráfico de sedimentación

Este gráfico nos sirve para ver como va bajando la varianza explicada de cada componente

```
plot(pca1,type="l")
```




Para este gráfico puede usar `plot(pca$sdev^2)` o `plot(pca)` ó también si le dan solo los valores propios hagalo de la siguiente forma:

```
plot(1:5,c(2.341, 0.958, 0.656, 0.603, 0.442),type = "l",ylab = "Valor propio") # la lista de valores p
```

El criterio usando el gráfico dice que se deben escoger la cantidad de componentes antes de que caiga abruptamente la variabilidad, en este caso solo podríamos escoger uno, ya que en el gráfico anterior después del primer valor la varianza se desploma.

Cuándo usamos los porcentajes la idea es que el PCA siempre represente al menos un 80% de la variabilidad, para este caso tenemos el siguiente resultado.

```
sum(lambda1[1:3])/length(lambda1)
```

```
## [1] 0.8646255
```

Usaríamos tres componentes en lugar de uno, lo que pasa es que con dos componente no puedo lograr el objetivo de hacer los gráfico más fácil.

**Si un componente tiene una varianza menor a uno, no se justifica incluirlo ya que una variable explica más que un componente, debido a que las variables como están estandarizas tienen varianza 1**. Normalmente esto ocurre porque las variables no se están tan correlacionadas entre sí.

### 11.0.1 Conclusión de estudio

Debido a que según el gráfico solo debemos quedarnos con un componente y con el criterio del porcentaje con tres componentes, lo cual no nos sirve para el análisis, por lo que aplicar PCA para este estudio no fue satisfactorio.

## 11.1 Reproducción de la matriz de correlaciones

Usando todos los componentes.

```
A=pca1$rot # 1. Extraer la matriz de vectores propios (eigenvectors o loadings)
L=diag(pca1$sdev**2) # 2. Construir la matriz diagonal de valores propios (eigenvalues o varianzas)
R1=A%*%L%*%t(A) #3. Aplicar el producto matricial de la descomposición espectral: R1 = A %*% L %*% t(A)
round(R1,2)
```

```
##            vallas saltoalto tiro car200 saltolargo javalina car800
## vallas       1.00      0.58 0.77   0.83       0.89     0.33   0.56
## saltoalto    0.58      1.00 0.46   0.39       0.66     0.35   0.15
## tiro         0.77      0.46 1.00   0.67       0.78     0.34   0.41
## car200       0.83      0.39 0.67   1.00       0.81     0.47   0.57
## saltolargo   0.89      0.66 0.78   0.81       1.00     0.29   0.52
## javalina     0.33      0.35 0.34   0.47       0.29     1.00   0.26
## car800       0.56      0.15 0.41   0.57       0.52     0.26   1.00
```

Verificación

```
round(R,2)
```

```
##            vallas saltoalto tiro car200 saltolargo javalina car800
## vallas       1.00      0.58 0.77   0.83       0.89     0.33   0.56
## saltoalto    0.58      1.00 0.46   0.39       0.66     0.35   0.15
## tiro         0.77      0.46 1.00   0.67       0.78     0.34   0.41
## car200       0.83      0.39 0.67   1.00       0.81     0.47   0.57
## saltolargo   0.89      0.66 0.78   0.81       1.00     0.29   0.52
## javalina     0.33      0.35 0.34   0.47       0.29     1.00   0.26
## car800       0.56      0.15 0.41   0.57       0.52     0.26   1.00
```

```
round(abs(R1-R),10)
```

```
##            vallas saltoalto tiro car200 saltolargo javalina car800
## vallas          0         0    0      0          0        0      0
## saltoalto       0         0    0      0          0        0      0
## tiro            0         0    0      0          0        0      0
## car200          0         0    0      0          0        0      0
## saltolargo      0         0    0      0          0        0      0
## javalina        0         0    0      0          0        0      0
## car800          0         0    0      0          0        0      0
```

Ahora con sólo dos componentes obtenemos una predicción de R y vea las discrepancias. Para facilidad vea las discrepancias con solo un decimal.

```
A2=pca1$rot[,1:2]
L2=diag(lambda1[1:2])
R2=A2%*%L2%*%t(A2)
round(R2-R,1)
```

```
##            vallas saltoalto tiro car200 saltolargo javalina car800
## vallas       -0.1       0.0  0.0    0.0        0.0      0.1    0.1
## saltoalto     0.0      -0.2  0.1    0.1        0.0      0.2   -0.1
```

```
## tiro          0.0       0.1 -0.3    0.1        0.0      0.1    0.1
## car200        0.0       0.1  0.1   -0.2        0.0     -0.1    0.1
## saltolargo    0.0       0.0  0.0    0.0       -0.1      0.2    0.1
## javalina      0.1       0.2  0.1   -0.1        0.2     -0.7   -0.1
## car800        0.1      -0.1  0.1    0.1        0.1     -0.1   -0.2
```

### 11.2 Obtención de correlaciones

Obtenemos la correlación entre cada variable original y Z1a.

```
cor(base2,Z1a)
```

```
##                  [,1]
## vallas     -0.9365076
## saltoalto  -0.6539754
## tiro       -0.8369090
## car200     -0.8880557
## saltolargo -0.9377059
## javalina   -0.5038399
## car800     -0.6298453
```

Obtenemos las correlaciones a partir del primer valor propio y el primer vector propio. Compare los resultados.

```
a1*sqrt(lambda1[1]) # a1 es el primer vector propio
```

```
##     vallas  saltoalto       tiro     car200 saltolargo   javalina     car800
## -0.9365076 -0.6539754 -0.8369090 -0.8880557 -0.9377059 -0.5038399 -0.6298453
```

## 12 Prueba de los componentes en una regresión

```
Z1=pca1$x[,1]
Z2=pca1$x[,2]
Y=base1$puntaje
mod=lm(Y~Z1+Z2)
summary(mod)
```

```
##
## Call:
## lm(formula = Y ~ Z1 + Z2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -233.905   -7.667    3.637   25.755   88.441
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 6154.125     11.981 513.665   <2e-16 ***
## Z1          -230.017      5.886 -39.080   <2e-16 ***
## Z2             6.373     12.908   0.494    0.627
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 58.69 on 21 degrees of freedom
## Multiple R-squared:  0.9864, Adjusted R-squared:  0.9851
## F-statistic: 763.8 on 2 and 21 DF,  p-value: < 2.2e-16
```

Observe que `Z1` es significativo y `Z2` no es significativo, recuerde que estos son variables así que puede ser que los resultados sean muy distintos para otros estudios.

## 12.1 Forma automática

```
step(mod)
```

```
## Start:  AIC=198.27
## Y ~ Z1 + Z2
##
##        Df Sum of Sq     RSS    AIC
## - Z2    1       840   73184 196.54
## <none>                72344 198.27
## - Z1    1   5261348 5333693 299.48
##
## Step:  AIC=196.54
## Y ~ Z1
##
##        Df Sum of Sq     RSS    AIC
## <none>                73184 196.54
## - Z1    1   5261348 5334533 297.48
```

```
##
## Call:
## lm(formula = Y ~ Z1)
##
## Coefficients:
## (Intercept)           Z1
##        6154         -230
```

## 12.2 Interpretaciones de regresión

Teneamos esta ecuación resultante del modelo anterior:

$$\hat{y} = 6154.13 - 230.02(-0.45 * vallas - 0.31 * \text{salto alto} - 0.4 * Tiro - 0.43 * car200 - 0.45 * \text{salto largo} + 0.24 * javalina - 0.3 * car800)$$

Multiplicamos los coeficientes de $Z_1$, por el coeficiente de $Z_1$

```
a = c(-0.45,-0.31,-0.40,-0.43,-0.45,0.24,-0.3)

-230.02*a
```

```
## [1] 103.5090  71.3062  92.0080  98.9086 103.5090 -55.2048  69.0060
```

$$\hat{y} = 6154.13 + 103.54 \text{ vallas}^s + 71.03 * \text{salto alto}^s + 92 * Tiro^s + 98.9 * car200^s + 103.5 * \text{salto largo}^s - 55.2 * javalina^s + 69 * car800^s$$

**Obtenemos al desviación estándar de vallas**

```
sd(base1$vallas)
```

```
## [1] 0.514564
```

Para este caso las $X's$ estaban estarizadas, pero la $Y$ no entonces la interpretación tiene la siguiente estructura:

*Por un aumento de 1 desviación estándar en la variable $X_j$, la respuesta promedio Y (aumenta/disminuye) en promedio $\beta_j^s$ unidades, manteniendo constantes los demás predictores*

Aplicando esa estructura a el ejemplo nos quedaría así:

*Por una disminución* (esto es por contexto) *de 0.51 segundos en la Vallas, la respuesta promedio aumenta.*

Recordar que las variables que se le dieron vuelta fueron `vallas`, `car200` y `car800`.

*Nota: El objetivo de PCA no es siempre hacer una regresión.*

# 13 Conglomerados

## 13.1 Distancias entre individuos

Dadas los valores de las variables para los individuos i-ésimo y j-ésimo, interesa tener una medida de la distancia multidimensional entre esos dos individuos.

Si se cuenta con n individuos, la matriz de distancias es de tamaño nxn, con ceros en la diagonal, esto independientemente de q (número de variables).

*A continuación se le presenta un ejemplo de la matriz de distancias*

$$D = \begin{pmatrix} 0 & d_{12} & \cdots & d_{1n} \\ d_{21} & 0 & \cdots & d_{2n} \\ \vdots & \vdots & \ddots & \\ d_{n1} & d_{n2} & & 0 \end{pmatrix}$$

La distancia es un concepto opuesto al de similitud, entre mayor similitud exista entre dos sujetos, menor es la distancia entre ellos y viceversa.

## 13.2 Distancia Euclidea

La distancia más común es la Euclídea, definida por:

$$d_{ij} = \sqrt{\sum_{k=1}^{q}(x_{ik} - x_{jk})^2} = \sqrt{(X_i - X_j)^T(X_i - X_j)}$$

donde $x_{ik}$ es el valor que toma el i-ésimo sujeto en la k-ésima variable, y $X_i$ es el vector de $q$ valores del i-ésimo sujeto:$X_i^T = (x_{i1}, \cdots, x_{ik}, \cdots, x_{iq})$.

En dos dimensiones, es la medida de la hipotenusa del triángulo rectángulo que se forma a partir de los dos puntos que se están comparando.

El cuadrado de la distancia Euclídea ($d_{ij}^2$) es otra medida de distancia que se puede usar en algunas ocasiones.

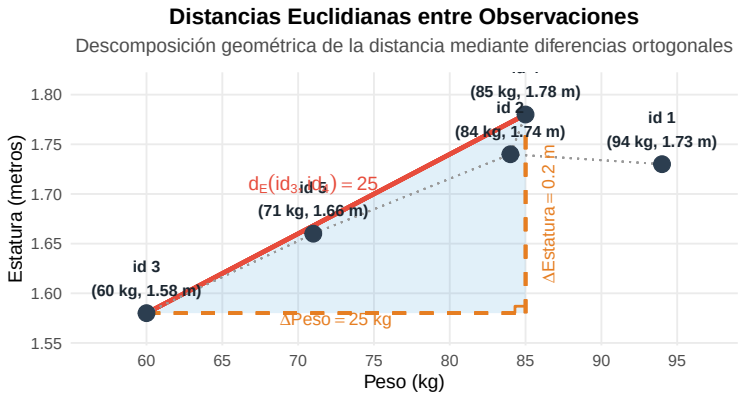
$$d_{ij}^2 = \sum_{k=1}^{q}(x_{ik} - x_{jk})^2$$

**Ejemplo aplicado**

Tenemos los siguientes datos tomados en el momento de la clase

| id | Peso | Estatura (metros) | Edad |
|---|---|---|---|
| 1 | 94 | 1.73 | 23.40 |

| id | Peso | Estatura (metros) | Edad |
|---|---|---|---|
| 2 | 84 | 1.74 | 21.90 |
| 3 | 60 | 1.58 | 21.60 |
| 4 | 85 | 1.78 | 20.04 |
| 5 | 71 | 1.66 | 25.30 |




Cálculamos las distancias entre los puntos

$$d(1,2) = \sqrt{(94-84)^2 + (1.73-1.74)^2} = \sqrt{10^2 + (-0.01)^2} = \sqrt{100.0001} \approx 10.0000$$
$$d(1,3) = \sqrt{(94-60)^2 + (1.73-1.58)^2} = \sqrt{34^2 + 0.15^2} = \sqrt{1156.0225} \approx 34.0003$$
$$d(1,4) = \sqrt{(94-85)^2 + (1.73-1.78)^2} = \sqrt{9^2 + (-0.05)^2} = \sqrt{81.0025} \approx 9.0001$$
$$d(1,5) = \sqrt{(94-71)^2 + (1.73-1.66)^2} = \sqrt{23^2 + 0.07^2} = \sqrt{529.0049} \approx 23.0001$$

$$d(2,3) = \sqrt{(84-60)^2 + (1.74-1.58)^2} = \sqrt{24^2 + 0.16^2} = \sqrt{576.0256} \approx 24.0005$$
$$d(2,4) = \sqrt{(84-85)^2 + (1.74-1.78)^2} = \sqrt{(-1)^2 + (-0.04)^2} = \sqrt{1.0016} \approx 1.0008$$
$$d(2,5) = \sqrt{(84-71)^2 + (1.74-1.66)^2} = \sqrt{13^2 + 0.08^2} = \sqrt{169.0064} \approx 13.0002$$

$$d(3,4) = \sqrt{(60-85)^2 + (1.58-1.78)^2} = \sqrt{(-25)^2 + (-0.20)^2} = \sqrt{625.0400} \approx 25.0008$$
$$d(3,5) = \sqrt{(60-71)^2 + (1.58-1.66)^2} = \sqrt{(-11)^2 + (-0.08)^2} = \sqrt{121.0064} \approx 11.0003$$

$$d(4,5) = \sqrt{(85-71)^2 + (1.78-1.66)^2} = \sqrt{14^2 + 0.12^2} = \sqrt{196.0144} \approx 14.0005$$

Podemos ver que la diferencias entre el peso en kilogramos, tiene mayor importancia al momento de hacer el cálculo, para solucionar este problema lo idea el estandarizar las variables.

## 13.3 Distancia de Mahalanobis

Esta medida de distancia incluye un procedimiento de estandarización dentro de la distancia Euclídea.

No sólo hace una estandarización en los datos escalándolos en términos de su desviación estándar sino que ajusta según las correlaciones entre variables:

$$d_{ij}^M = \sqrt{(X_i - X_j)^T S^{-1} (X_i - X_j)}$$

donde $S$ es la matriz de covariancias de dimensión $q \times q$

Es invariante frente a transformaciones lineales de las variables.

Toma en cuenta las correlaciones entre variables.

No aumenta por el simple hecho de aumentar el número de variables, sino que aumentará cuando las nuevas variables den nueva información.

**Aplicación basada en el ejemplo anterior**

$$
\begin{aligned}
d_{35}^{M} &= \sqrt{(X_3 - X_5)^\top \mathbf{S}^{-1} (X_3 - X_5)} \\
&= \sqrt{\begin{pmatrix} 60-71 & 1.58-1.66 \end{pmatrix} \begin{bmatrix} 177.7000 & 0.9320 \\ 0.9320 & 0.00622 \end{bmatrix}^{-1} \begin{pmatrix} 60-71 \\ 1.58-1.66 \end{pmatrix}} \\
&= \sqrt{\begin{pmatrix} -11 & -0.08 \end{pmatrix} \begin{bmatrix} 0.0263 & -3.9380 \\ -3.9380 & 750.8345 \end{bmatrix} \begin{pmatrix} -11 \\ -0.08 \end{pmatrix}} \\
&= \sqrt{\begin{pmatrix} 0.0259 & -16.7491 \end{pmatrix} \begin{pmatrix} -11 \\ -0.08 \end{pmatrix}} \\
&= \sqrt{(-11)(0.0259) + (-0.08)(-16.7491)} \\
&= \sqrt{-0.2849 + 1.3399} \\
&= \sqrt{1.0550} \approx \mathbf{1.0269}
\end{aligned}
$$

```r
X = as.matrix(datos[, c("Peso", "Estatura")])
n = nrow(X)
S = cov(X)
S_inv = solve(S)


# Matriz de distancias de Mahalanobis par a par
D_M <- matrix(0, nrow = n, ncol = n)
for (i in 1:n) {
  for (j in 1:n) {
    diff <- X[i, ] - X[j, ]
    D_M[i, j] <- sqrt(t(diff) %*% S_inv %*% diff)
  }
}
rownames(D_M) <- colnames(D_M) <- paste0("id_", 1:n)

print("Matriz de Distancias de Mahalanobis (pares):")
```

```
## [1] "Matriz de Distancias de Mahalanobis (pares):"
```

```r
print(round(D_M, 4))
```

```
##        id_1   id_2   id_3   id_4   id_5
## id_1 0.0000 1.8684 2.6660 2.7477 2.2140
## id_2 1.8684 0.0000 2.0287 0.9553 1.0276
## id_3 2.6660 2.0287 0.0000 2.6607 1.0269
## id_4 2.7477 0.9553 2.6607 0.0000 1.6527
## id_5 2.2140 1.0276 1.0269 1.6527 0.0000
```

### 13.3.1 Forma automática

```
#install.packages("biotools")
library(biotools)
dM=D2.dist(X,S) #Esto cálcula la distancia de Mahalanobis al Cuadrado
round(sqrt(dM),4)
```

```
##        1      2      3      4
## 2 1.8684
## 3 2.6660 2.0287
## 4 2.7477 0.9553 2.6607
## 5 2.2140 1.0276 1.0269 1.6527
```

Verifique la distancia $d_{35}^{M}$ es 1.0269

## 13.4 Distancia City-block (Manhattan)

En esta medida se usa el valor absoluto de las diferencias en lugar de su cuadrado:

$$d_{ij}^{(1)} = \sum_{k=1}^{q} |x_{ik} - x_{jk}|$$

El nombre nace de la idea de moverse en una ciudad como Manhattan pues, aunque la distancia más corta sería la Euclidea, ésta implica pasar a través de los edificios, mientras que esta distancia asemeja a caminar por las aceras para llegar de un punto a otro.

Es más robusta que la distancia euclídea debido a que no eleva al cuadrado las diferencias.

Tiene la restricción de que asume que las variables no están correlacionadas.

**Aplicación basada en el ejemplo anterior**

$$\begin{aligned}
d(1,2) &= |94-84| + |1.73-1.74| = 10 + 0.01 = \mathbf{10.01} \\
d(1,3) &= |94-60| + |1.73-1.58| = 34 + 0.15 = \mathbf{34.15} \\
d(1,4) &= |94-85| + |1.73-1.78| = 9 + 0.05 = \mathbf{9.05} \\
d(1,5) &= |94-71| + |1.73-1.66| = 23 + 0.07 = \mathbf{23.07} \\
d(2,3) &= |84-60| + |1.74-1.58| = 24 + 0.16 = \mathbf{24.16} \\
d(2,4) &= |84-85| + |1.74-1.78| = 1 + 0.04 = \mathbf{1.04} \\
d(2,5) &= |84-71| + |1.74-1.66| = 13 + 0.08 = \mathbf{13.08} \\
d(3,4) &= |60-85| + |1.58-1.78| = 25 + 0.20 = \mathbf{25.20} \\
d(3,5) &= |60-71| + |1.58-1.66| = 11 + 0.08 = \mathbf{11.08} \\
d(4,5) &= |85-71| + |1.78-1.66| = 14 + 0.12 = \mathbf{14.12}
\end{aligned}$$

*Aplicación en r*

```
# Distancia de Manhattan para Peso y Estatura
D_manhattan_2v <- dist(datos[, c("Peso", "Estatura")], method = "manhattan")
print(as.matrix(round(D_manhattan_2v, 2)))
```

```
##       1     2     3     4     5
## 1  0.00 10.01 34.15  9.05 23.07
## 2 10.01  0.00 24.16  1.04 13.08
## 3 34.15 24.16  0.00 25.20 11.08
## 4  9.05  1.04 25.20  0.00 14.12
```

```
## 5 23.07 13.08 11.08 14.12  0.00
```

```
# Distancia de Manhattan completa (3 variables)
D_manhattan_3v <- dist(datos, method = "manhattan")
print(as.matrix(round(D_manhattan_3v, 2)))
```

```
##       1     2     3     4     5
## 1  0.00 11.01 36.15 12.05 27.07
## 2 11.01  0.00 25.16  3.04 16.08
## 3 36.15 25.16  0.00 26.20 13.08
## 4 12.05  3.04 26.20  0.00 15.12
## 5 27.07 16.08 13.08 15.12  0.00
```

### 13.4.1 Nota importante de las distancias

Puede ser que usando algunos de los métodos, las agrupaciones que se estén realizando, por lo que dependiendo del caso se debe escoger el tipo de distancia que tenga más sentido. De modo que los grupos sean muy parecidos dentro de ellos, pero muy desiguales entre los grupos existentes.

# 14 Ejemplo de aplicación con temperaturas de diferentes cantones de Costa Rica

```
load("Temp_Cantones.Rdata")
```

Observe la estrucutura de la base

```
head(base)
```

```
## # A tibble: 6 x 4
##   canton   anio mes   temp_c
##   <chr>   <dbl> <fct>  <dbl>
## 1 Cartago  2025 JAN     19.9
## 2 Cartago  2025 FEB     20.0
## 3 Cartago  2025 MAR     20.7
## 4 Cartago  2025 APR     20.8
## 5 Cartago  2025 MAY     21.6
## 6 Cartago  2025 JUN     21.1
```

Para solucionar este problema debemos pivotear la base de datos con el siguiente código:

```
library(dplyr)
library(tidyr)
library(tibble)
base2 = base %>%
  dplyr::select(canton, mes, temp_c) %>%
  pivot_wider(
    names_from = mes,
    values_from = temp_c
  ) %>%
  as.data.frame() %>%
  column_to_rownames(var = "canton")
```

```
head(base2)
```

```
##                JAN   FEB   MAR   APR   MAY   JUN   JUL   AUG   SEP   OCT   NOV
## Cartago      19.89 19.99 20.68 20.78 21.60 21.11 20.88 20.94 21.16 20.81 20.39
```

```
## Corredores    26.27 26.87 27.73 27.65 26.58 26.04 26.47 26.18 25.88 25.28 25.61
## Limón         24.36 24.35 24.75 24.92 25.62 25.09 25.12 25.17 25.34 25.03 24.76
## Nicoya        27.43 28.63 29.45 29.52 28.28 27.33 27.92 27.64 27.09 26.47 26.85
## Puntarenas    26.59 27.40 28.57 28.51 27.66 26.82 27.40 27.20 26.72 25.95 26.40
## Pérez Zeledón 20.97 21.27 22.10 22.21 22.23 21.75 21.87 21.75 21.68 21.17 21.16
##                 DEC
## Cartago       19.67
## Corredores    25.76
## Limón         24.37
## Nicoya        26.96
## Puntarenas    26.28
## Pérez Zeledón 20.78
```

### 14.0.1 Cálculo distancia euclidiana

*A continuación se le va a proporcionar la distancia euclideana entre dos individuos*

```
x1=base2[1,]
x2=base2[2,]
x12=as.matrix(x1-x2,ncol=1)
sqrt(x12%*%t(x12))
```

```
##          Cartago
## Cartago 19.98362
```

### 14.0.2 ¿Que sucede si transponemos la base?

```
base3 = t(base2)
head(base3)
```

```
##     Cartago Corredores Limón Nicoya Puntarenas Pérez Zeledón Santa Cruz
## JAN   19.89      26.27 24.36  27.43      26.59         20.97      26.59
## FEB   19.99      26.87 24.35  28.63      27.40         21.27      28.19
## MAR   20.68      27.73 24.75  29.45      28.57         22.10      29.72
## APR   20.78      27.65 24.92  29.52      28.51         22.21      29.78
## MAY   21.60      26.58 25.62  28.28      27.66         22.23      28.63
## JUN   21.11      26.04 25.09  27.33      26.82         21.75      27.20
##     Talamanca Upala Vázquez de Coronado
## JAN     20.52 25.34               22.86
## FEB     20.64 26.31               23.09
## MAR     21.25 28.21               24.05
## APR     21.38 28.23               24.31
## MAY     22.15 27.77               24.15
## JUN     21.71 26.26               23.55
```

En el cálculo de la distancia lo que cambia es la posición de los datos y el lugar de la matriz transpuesta en la multiplicación de matrices

```
x1=as.numeric(base3[,1])
x2=as.numeric(base3[,2])
x12=as.matrix(x1-x2,ncol=1)
sqrt(t(x12)%*%x12)
```

```
##          [,1]
## [1,] 19.98362
```

*A continuación se le presenta una forma automática de calculare todas las ditancias por medio de la función* `dist()`

```
(d=dist(base2))
```

```
##                       Cartago Corredores     Limón    Nicoya Puntarenas
## Corredores          19.983618
## Limón               14.732074   5.831569
## Nicoya              24.992805   5.054157 10.641414
## Puntarenas          22.582608   2.749109  8.204767  2.488715
## Pérez Zeledón        3.396145  16.672942 11.592886 21.686860  19.291008
## Santa Cruz          24.327885   4.680833 10.151345  1.493352   2.247221
## Talamanca            2.126006  17.883582 12.611189 22.893610  20.479187
## Upala               20.422030   2.076126  6.326302  4.892852   2.653356
## Vázquez de Coronado  9.772712  10.237846  5.271442 15.236430  12.820729
##                     Pérez Zeledón Santa Cruz Talamanca     Upala
## Corredores
## Limón
## Nicoya
## Puntarenas
## Pérez Zeledón
## Santa Cruz              21.032432
## Talamanca                1.541006  22.242704
## Upala                   17.162613   4.013365 18.334350
## Vázquez de Coronado      6.491849  14.600449  7.678086 10.723656
```

Observe la clase del objeto `d`

```
class(d)
```

```
## [1] "dist"
```

### 14.0.3 Extración de valores de un objeto dist()

```
d2= as.matrix(d) # Pasamos el objeto dist a una matriz
d2sup = d2
# Para evitar los ceros en la diagonal extraemos la matriz trangular superior
# Se extrae la matriz trangular superior y reemplazan los otros valores con NA
d2sup[lower.tri(d2, diag = TRUE)] <- NA
```

*Cantones con Mayor distancia*

```
#Extraemos la posición de la distancia más grande
max_pos = which(d2sup == max(d2sup, na.rm = TRUE), arr.ind = TRUE)
#Obtenemos los nombres de las variables de forma automática
cantones_max <- data.frame(
  Canton_1 = rownames(d2)[max_pos[, 1]],
  Canton_2 = colnames(d2)[max_pos[, 2]],
  Distancia = d2[max_pos]
)
cantones_max
```

```
##   Canton_1 Canton_2 Distancia
## 1  Cartago   Nicoya   24.9928
```

*Cantones con menor distancia*

```r
min_pos = which(d2sup == min(d2sup, na.rm = TRUE), arr.ind = TRUE)
cantones_min = data.frame(
  Canton_1 = rownames(d2)[min_pos[, 1]],
  Canton_2 = colnames(d2)[min_pos[, 2]],
  Distancia = d2[min_pos]
)
cantones_min
```

```
##   Canton_1   Canton_2 Distancia
## 1   Nicoya Santa Cruz  1.493352
```

#### 14.0.4 Cálculo distancia de Mahalannobis

```r
library(biotools)
S=var(base2)
dM=D2.dist(base2,S)
# DA ERROR PORQUE S NO SE PUEDE INVERTIR
```

#### 14.0.5 Cálculo distancia de Manhattan

```r
dMan=dist(base2,method = "manhattan")
as.matrix(dMan)[1,2]
```

```
## [1] 68.42
```

### 14.1 Creación de dendogramas y distancias entre grupos (linkage)

A partir de una matriz de distancias entre individuos se puede derivar el concepto de distancia entre dos grupos

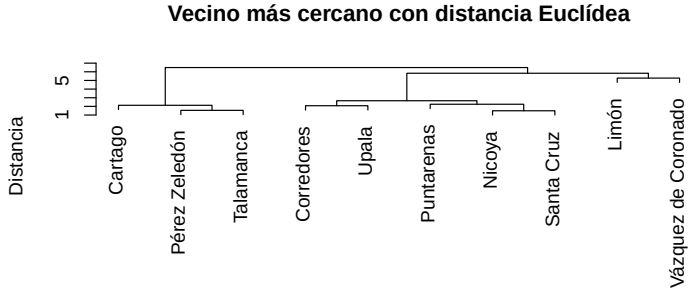
#### 14.1.1 K-vecino más cercano (singke linkage)

$$\delta(A,B) = \min_{\substack{x_i \in A \\ x_j \in B}} d(x_i, x_j)$$

Para crear un dendograma de este tipo de distncia se debe indicar con `method = "single"`.

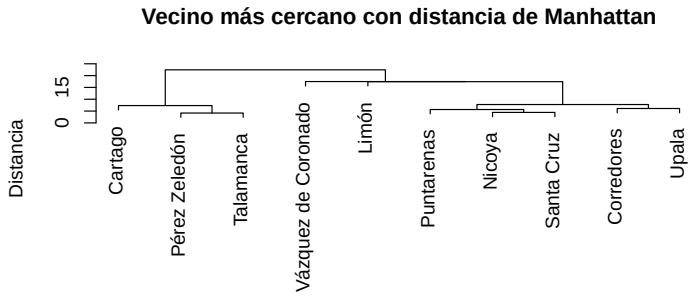
```r
cs = hclust(d,method = "single")
```

```r
plot(cs,xlab="",main="Vecino más cercano con distancia Euclídea",
     ylab="Distancia",sub="")
```



```
csM = hclust(dM,method = "single")
plot(csM,xlab="",main="Vecino más cercano con distancia de Mahalannobis",
     ylab="Distancia",sub="")
```

```
csMan = hclust(dMan,method = "single")
plot(csMan,xlab="",main="Vecino más cercano con distancia de Manhattan",
     ylab="Distancia",sub="")
```

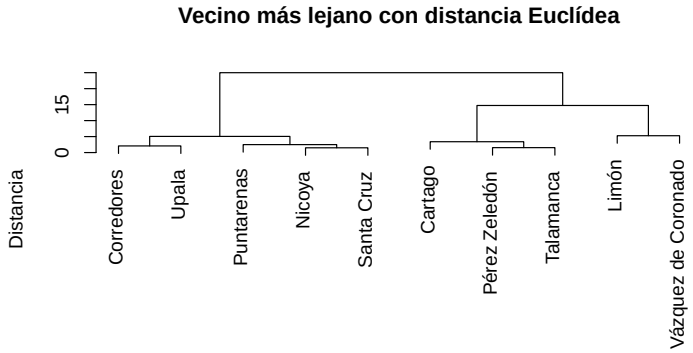



### 14.1.2 K-vecino más lejano (complete linkage)

$$\delta(A, B) = \max_{\substack{x_i \in A \\ x_j \in B}} d(x_i, x_j)$$

Para crear un dendograma de este tipo de distncia se debe indicar con `method = "complete"`.

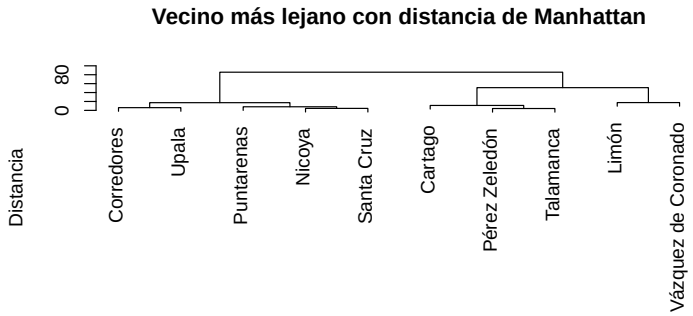
```
c.com = hclust(d,method = "complete")
plot(c.com, xlab = "", main = "Vecino más lejano con distancia Euclídea",
     ylab = "Distancia", sub = "")
```




```
cM.com = hclust(dM,method = "complete")
plot(cM.com,xlab="",main="Vecino más lejano con distancia de Mahalannobis",
     ylab="Distancia",sub="")
```

```
cMan.com = hclust(dMan,method = "complete")
plot(cMan.com,xlab="",main="Vecino más lejano con distancia de Manhattan",
```

```
     ylab="Distancia",sub="")
```




### 14.1.3 Salto promedio

Es el promedio de todas las distancias mínimas.

$$\delta(A,B) = \frac{1}{n_A n_B} \sum_{x_i \in A, x_j \in B} d(x_i, x_j)$$

```
c.av = hclust(d,method = "average")
plot(c.av,xlab="",main="Salto promedio con distancia Euclidea",
     ylab="Distancia",sub="")
```




```
cM.av = hclust(dM,method = "average")
plot(cM.av,xlab="",main="Salto promedio con distancia de Mahalannobis",
     ylab="Distancia",sub="")
```

```
cMan.av = hclust(dMan,method = "average")
plot(cMan.av,xlab="",main="Salto promedio con distancia de Manhattan",
     ylab="Distancia",sub="")
```

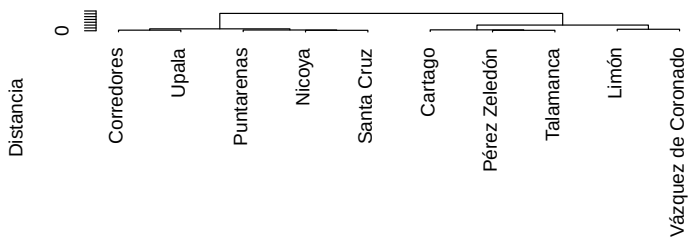



### 14.1.4 Distancia de Ward

$$\delta(A,B) = \frac{n_A n_B}{n_A + n_B} \|g_A - g_B\|^2$$

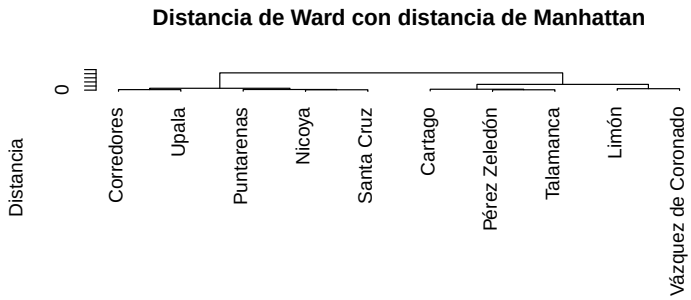
donde: $g_A$ es el centroide del clúster A

```
c.w = hclust(d,method = "ward.D")
plot(c.w,xlab="",main="Distancia de Ward con distancia Euclidea",
     ylab="Distancia",sub="")
```




```
cM.w = hclust(dM,method = "ward.D")
plot(cM.w,xlab="",main="Distancia de Ward con distancia de Mahalannobis",
     ylab="Distancia",sub="")
```

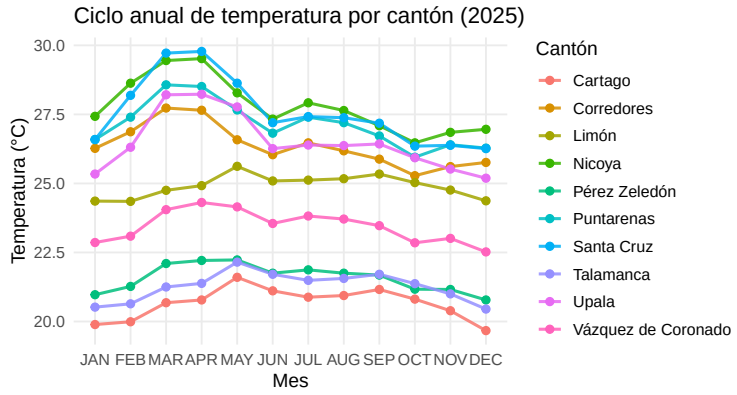
```
cMan.w = hclust(dMan,method = "ward.D")
plot(cMan.w,xlab="",main="Distancia de Ward con distancia de Manhattan",
     ylab="Distancia",sub="")
```




## 14.2 ¿Los dos cantones más correlacionados son los cantones que menor distancia tienen?

*A continuación se le presenta un gráfico con las temperaturas de todos los meses por cantón*

```
 #datos largos son los datos pivioteados
ggplot(datos_largos, aes(x = mes, y = temp_c, group = canton, color = canton)) +
  geom_line(linewidth = 0.9, alpha = 0.85) +
  geom_point(size = 2) +
  labs(
    title = "Ciclo anual de temperatura por cantón (2025)",
    x = "Mes",
    y = "Temperatura (°C)",
    color = "Cantón"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    legend.position = "right"
  )
```



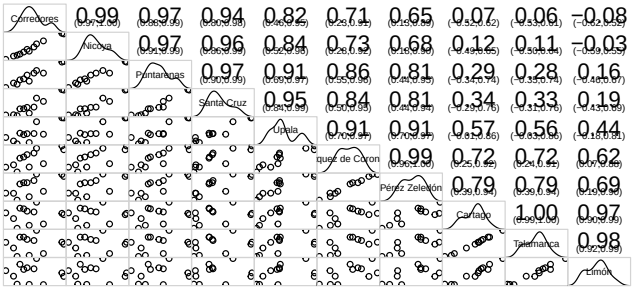

Recordermos que la menor distancia entre cantones se encontraba entre:

```
cantones_min #Buscar en las páginas de arriba como se obtuvo este resultado
```

```
##   Canton_1   Canton_2 Distancia
## 1   Nicoya Santa Cruz  1.493352
```

*A continuación se le presenta un gráfico de la matriz de correlaciones*

```
corrgram(base3, lower.panel=panel.pts, upper.panel=panel.conf,
         diag.panel=panel.density,order=T)
```




Basado en el gráfico anterior y con la función `cor()` obtenemos la mayor correlación entre cantones.

```
R = cor(base3)
Rsup = R
# Se extrae la matriz trangular superior y reemplazan los otros valores con NA
Rsup[lower.tri(R, diag = TRUE)] <- NA
max_r = which(Rsup == max(Rsup, na.rm = TRUE), arr.ind = TRUE)
```

```
#Obtenemos los nombres de las variables de forma automática
cantones_maxr <- data.frame(
  Canton_1 = rownames(R)[max_r[, 1]],
  Canton_2 = colnames(R)[max_r[, 2]],
  Correlación = R[max_r]
)
```

Basado en el proceso anterior tenemos los siguientes `dataframe()`

```
cantones_min # el código se encuentra más arriba
```

```
##   Canton_1   Canton_2 Distancia
## 1   Nicoya Santa Cruz  1.493352
```

```
cantones_maxr
```

```
##   Canton_1  Canton_2 Correlación
## 1  Cartago Talamanca   0.9973338
```

Obtenemos los nombres para poder graficarlos en un solo gráfico

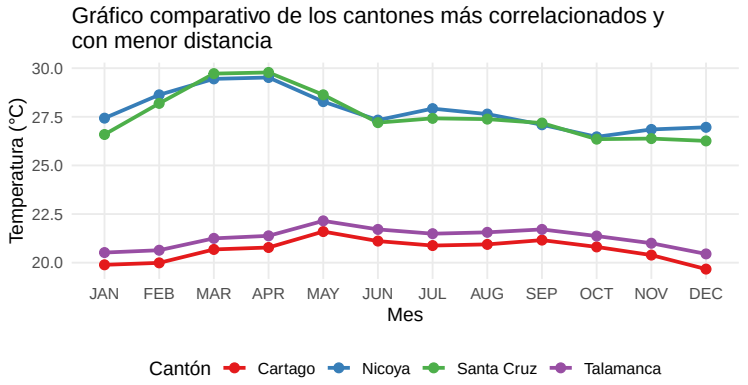
```
d1 = cantones_min[1,1] #Nicoya
d2 = cantones_min[1,2] #Santa Cruz
r1 = cantones_maxr[1,1] #Cartago
r2 = cantones_maxr[1,2] #Talamanca
```

```
# Unir cantones a graficar (sin duplicados)
cantones_a_graficar <- unique(c(d1,d2,r1,r2))

# Filtrar y pivotar a formato largo
datos_lineas <- base2 %>%
  as.data.frame() %>%
```

```r
  rownames_to_column(var = "canton") %>%
  filter(canton %in% cantones_a_graficar) %>%
  pivot_longer(
    cols = JAN:DEC,
    names_to = "mes",
    values_to = "temp_c"
  ) %>%
  mutate(
    mes = factor(mes, levels = c("JAN", "FEB", "MAR", "APR", "MAY", "JUN",
                                 "JUL", "AUG", "SEP", "OCT", "NOV", "DEC"))
  )

# Gráfico lineal comparativo
ggplot(datos_lineas, aes(x = mes, y = temp_c, group = canton, color = canton)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.5) +
  scale_color_brewer(palette = "Set1") +
  labs(
  title = "Gráfico comparativo de los cantones más correlacionados y \ncon menor distancia",
    x = "Mes",
    y = "Temperatura (°C)",
    color = "Cantón"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    legend.position = "bottom"
  )
```




Se puede ver en el anterior gráfico que los cantones de Cartago y Talamanca, tienen el mismo comportamiento, pero no son tan cercanos. Por otro lado, los cantones Santa Cruz y Nicoya son muy cercanos, pero su comportamiento si es más distinto, estos dos tipos de distancia nos sirven dependiendo del tipo de pregunta que buscamos resolver.

A partir de una matriz de correlaciones $R$, se puede obtener una matriz de distancias de la siguiente forma:

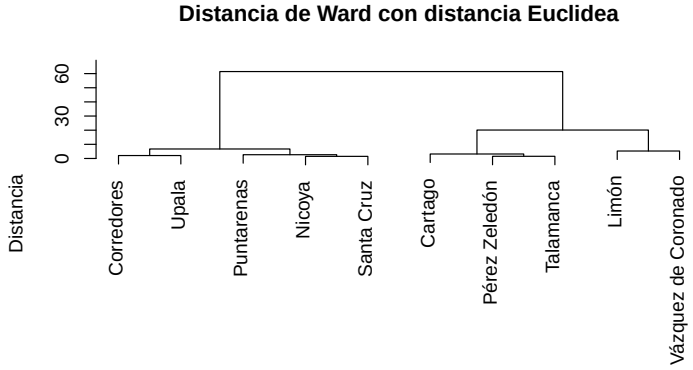
$$D = 1 - R$$

*A continuación se le presenta la forma de hacerlo en R*

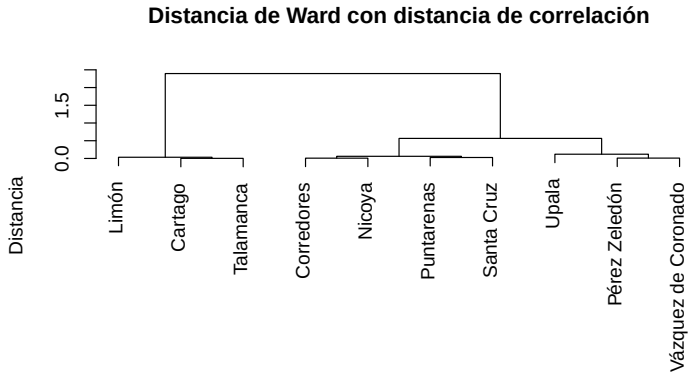
```
D = 1-R
dr = as.dist(D)
```

Hacemos un dendograma y lo comparamos con el que se obtiene con la distancia euclidea. Es importante resaltar que el mejor tipo de distancia es la distancia de `Ward`, por esa razón es que los siguientes gráficos van con esta distancia. *Los demás tipos de distancia se encuentran en la anterior sección*

```
c.w = hclust(d,method = "ward.D")
plot(c.w,xlab="",main="Distancia de Ward con distancia Euclidea",
     ylab="Distancia",sub="")
```




```
c.w = hclust(dr,method = "ward.D")
plot(c.w,xlab="",main="Distancia de Ward con distancia de correlación",
     ylab="Distancia",sub="")
```



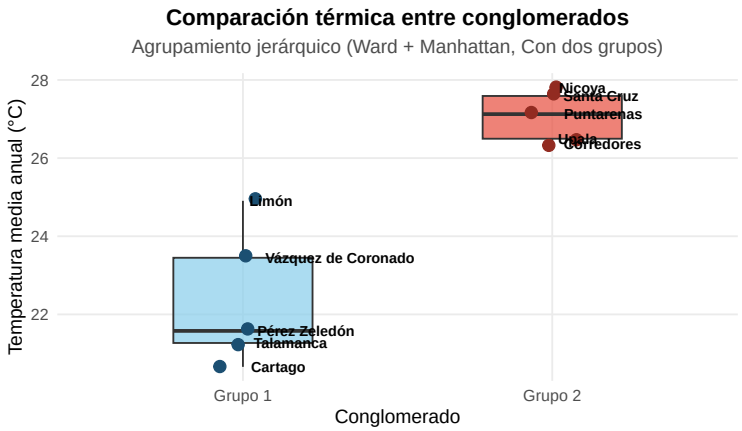

```
grupos <- cutree(cMan.w, k = 2)
# Ver la asignación de cantones por grupo en consola
print(split(names(grupos), grupos))
```

```
## $`1`
## [1] "Cartago"             "Limón"               "Pérez Zeledón"
```

```
## [4] "Talamanca"           "Vázquez de Coronado"
##
## $`2`
## [1] "Corredores" "Nicoya"     "Puntarenas" "Santa Cruz" "Upala"
```

```r
# 2. Preparar el data frame calculando la temperatura media anual por cantón
datos_cajas <- as.data.frame(base2) %>%
  rownames_to_column(var = "canton") %>%
  mutate(
    Grupo = factor(paste("Grupo", grupos[canton])),
    temp_media = rowMeans(across(JAN:DEC))
  )

# 3. Boxplot con exactamente dos cajas
ggplot(datos_cajas, aes(x = Grupo, y = temp_media, fill = Grupo)) +
  geom_boxplot(
    width = 0.45,
    alpha = 0.7,
    outlier.shape = NA,
    show.legend = FALSE
  ) +
  # Puntos observados (cantones) con dispersión horizontal leve
  geom_jitter(
    width = 0.08,
    size = 3.2,
    aes(color = Grupo),
    show.legend = FALSE
  ) +
  # Etiquetas con el nombre del cantón
  geom_text(
    aes(label = canton),
    hjust = -0.15,
    vjust = 0.5,
    size = 3.3,
    fontface = "bold"
  ) +
  scale_fill_manual(values = c("Grupo 1" = "skyblue", "Grupo 2" = "#e74c3c")) +
  scale_color_manual(values = c("Grupo 1" = "#1b4f72", "Grupo 2" = "#922b21")) +
  labs(
    title = "Comparación térmica entre conglomerados",
    subtitle = "Agrupamiento jerárquico (Ward + Manhattan, Con dos grupos)",
    x = "Conglomerado",
    y = "Temperatura media anual (°C)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5, color = "gray30", margin = margin(b = 10)),
    panel.grid.minor = element_blank()
  )
```




## 14.3 ¿Que ocurre cuando tenemos variables categóricas?

Nos podemos ayudar con la función `daisy()`. La función daisy() (del paquete cluster) calcula matrices de disimilitud para observaciones con variables de tipo mixto (cuantitativas, nominales, ordinales y binarias) aplicando principalmente el coeficiente generalizado de Gower, el cual proyecta la diferencia de cada variable al intervalo $[0, 1]$ (dividiendo entre el rango empírico en numéricas, aplicando coincidencia booleana en nominales, rangos normalizados en ordinales y tratamiento asimétrico en binarias) para luego promediar esas distancias parciales; esto permite medir la separación entre individuos heterogéneos sin distorsionar la geometría de los datos mediante one-hot encoding, produciendo un objeto de clase dist directamente compatible con métodos no euclídeos como PAM (Partitioning Around Medoids) o agrupamiento jerárquico.

Cuando se tienen tanto variables métricas como variables nominales y/o ordinales se recomienda usar la distancia de Gower:

$$d_{ij}^{G} = \frac{1}{q}\left[\sum_{h=1}^{p_1} \frac{|x_{ih} - x_{jh}|}{R_h} + \beta_{ij}\right]$$

donde:

- $R_h$ : Rango de la h-ésima variable cuantitativa
- $\beta_{ij}$ : número de variables cualitativas que no son idénticas
- $q$ : número total de variables

### 14.3.1 Ejemplo de las universidades

Se cuenta con información de un gran número de universidades de los E.U.A de la edición de 1995 del US News and World Report.

Variables de tipo continuo:

- Tasa de aceptación (número aceptados / número aplicaciones)
- Costo de la matrícula para estudiantes de otros estados
- Número de nuevos estudiantes
- Tasa de graduación

Variables de tipo categórico (binarias):

- Tipo de universidad (pública o privada)
- Elite: si más del 50% de los estudiantes provienen de colegios del top-10%.

*A continuación se le presenta el formato de la base de datos del ejemplo*

| Universidad / Institución | tasa.acept | costo | nuevos | tasa.grad | privado | elite |
|---|---|---|---|---|---|---|
| Abilene Christian University | 0.74 | 7440 | 721 | 60 | Yes | No |
| Adelphi University | 0.88 | 12280 | 512 | 56 | Yes | No |
| Adrian College | 0.77 | 11250 | 336 | 54 | Yes | No |
| Agnes Scott College | 0.84 | 12960 | 137 | 59 | Yes | Yes |
| Alaska Pacific University | 0.76 | 7560 | 55 | 15 | Yes | No |
| Albertson College | 0.82 | 13500 | 158 | 55 | Yes | No |

Para el cálculo de la distancia de Gower se necesitan los rangos de las variables continuas:

*A continuación se le presenta los rangos de cada una de las varibles del ejemplo*

*Nota:* Recuerde que el rango es la diferencia entre el valor máximo y el valor mínimo

| tasa.acept | costo | nuevos | tasa.grad |
|---|---|---|---|
| 0.846 | 19360 | 6357 | 108 |

*Rangos en cada uno de las universidades*

| Universidad / Institución | tasa.acept | costo | nuevos | tasa.grad |
|---|---|---|---|---|
| Abilene Christian University | 0.74 | 7440 | 721 | 60 |
| Adelphi University | 0.88 | 12280 | 512 | 56 |

*Aplicamos la fórmula para la distancia de Gower*

$$\sum_{h=1}^{4} \frac{|x_{1h} - x_{2h}|}{R_h} = \frac{0.74 - 0.88}{0.846} + \frac{7440 - 12280}{19360} + \frac{721 - 512}{6357} + \frac{60 - 56}{108}$$

$$= \frac{0.14}{0.846} + \frac{4840}{19360} + \frac{209}{6357} + \frac{4}{108}$$

$$\approx 0.1655 + 0.2500 + 0.0329 + 0.0370$$

$$= 0.4854 \quad (\approx 0.483)$$

Observe que en este caso la que aporta más son la tasas de acptación y el costo debido a que tienen un aporte de 0.16 y 0.25 respectivamente.

*Se tiene la siguiente información de las dos universidad*

| Universidad / Institución | privado | elite |
|---|---|---|
| Abilene Christian University | Yes | No |
| Adelphi University | Yes | No |

En este caso como ambas variables cualitativas son cero entonces $\beta_{12} = 0$

$$d_{12}^{G} = \frac{1}{q}\left[\sum_{h=1}^{4}\frac{|x_{1h} - x_{2h}|}{R_h} + \beta_{12}\right] = \frac{1}{6}[0.483 + 0] = 0.0805$$

Luego de obtener este resultado lo tenemos que visualizar en un dendograma.

### 14.3.2 Ejemplo de universidades aplicado en r

```
head(universidades)
```

```
##                   institucion tasa.acept costo nuevos tasa.grad privado elite
## 1 Abilene Christian University       0.74  7440    721        60     Yes    No
## 2           Adelphi University       0.88 12280    512        56     Yes    No
## 3              Adrian College       0.77 11250    336        54     Yes    No
## 4          Agnes Scott College       0.84 12960    137        59     Yes   Yes
## 5    Alaska Pacific University       0.76  7560     55        15     Yes    No
## 6            Albertson College       0.82 13500    158        55     Yes    No
```

```
# 1. Base principal
base = universidades

# 2. Selección de las filas que componen los dos grupos iniciales (3 por grupo)
base1 = base[c(1, 2, 3, 4, 5, 6), ]

# 3. Cálculo de la matriz de disimilitud usando la distancia de Gower
library(cluster)
d = daisy(base1, metric = "gower")
D = as.matrix(d)

# 4. Submatrices de distancias internas para el Grupo 1 (filas 1:3) y Grupo 2 (filas 4:6)
d1 = D[1:3, 1:3]
d2 = D[4:6, 4:6]
```

Se determinan los medoides de ambos grupos

```
# 5. Distancia promedio hacia los otros miembros del grupo (n - 1 = 2)
# Distancias promedio de cada grupo
dist_prom_g1 = apply(d1, 2, sum) / 2
dist_prom_g2 = apply(d2, 2, sum) / 2

# Identificar automáticamente el nombre del medoide
medoide_g1 = names(which.min(dist_prom_g1))
medoide_g2 = names(which.min(dist_prom_g2))

# Imprimir los resultados
medoide_g1
```

```
## [1] "3"
```

```
medoide_g2
```

```
## [1] "6"
```

Se calcula la distancia de cada punto respecto a los 2 medoides y decida cuáles deben cambiar de grupo

```
D[c(3,6),]
```

```
##           1         2         3         4         5         6
## 3 0.3649157 0.3234843 0.0000000 0.4560126 0.4241326 0.2882742
## 6 0.5039836 0.3119493 0.2882742 0.3360552 0.4931876 0.0000000
```

En el grupo 1 deben quedarse son `1,3,5` y en el grupo 2 `2,4,6`

La distancia entre los dos grupos originales con vecino-más lejano

```
# Definir los vectores con los índices de cada grupo
g1 = c(1, 3, 5)
g2 = c(2, 4, 6)

# Extraer el bloque de distancias cruzadas entre G1 y G2
dist_cruzadas = D[g1, g2]

# Distancia por vecino más lejano (valor máximo)
dist_vecino_lejano = max(dist_cruzadas)

# Visualizar la submatriz y el resultado
dist_cruzadas
```

```
##           2         4         6
## 1 0.4573404 0.6463251 0.5039836
## 3 0.3234843 0.4560126 0.2882742
## 5 0.6047597 0.6519169 0.4931876
```

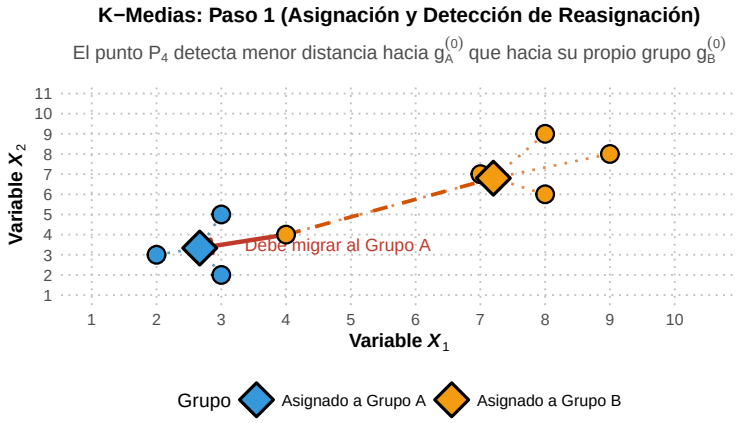
```
dist_vecino_lejano
```

```
## [1] 0.6519169
```

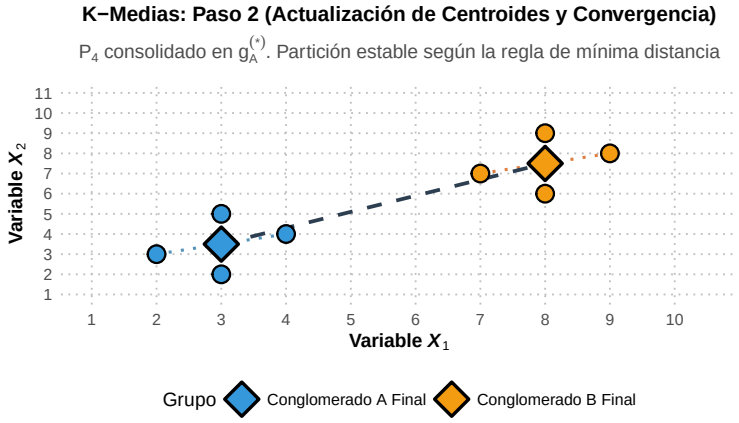
# 15 Método de k-medias y k-mediodes

## 15.1 k-medias

Si tenemos dos variables numéricas y ubicamos sus centroides, luego de eso cálculamos su distancia y obtenemos el siguiente posible resultado:




Luego se hace esa reasignación y se llega al siguiente resultado




Para un número determinado $k$ (grupos) se busca la partición que minimiza algún criterio de decisión, para el cual tener valores más bajos es una indicación de una buena solución.

El criterio más común es la suma de cuadrados dentro de grupo ($SCDG$). Dentro del j-ésimo grupo se obtiene $SCDG_j$ como la suma de las distancias euclídeas cuadráticas de cada punto en el j-ésimo cluster respecto al centroide de ese clúster.

La SCDG total es simplemente la suma de las $SCDG_j$

Si se intenta considerar todas las posibles particiones del conjunto de datos en $k$ grupos se puede llegar a números extremadamente altos de posibilidades, lo cual hace el problema computacionalmente impráctico.

### 15.1.1 Algoritmo 1

Se toma una partición arbitraria en k clústers que puede venir del resultado de otro método de agrupamiento.

*A continuación se le da la serie de pasos para este algoritmo*

1. Se calcula la SCDG dela partición inicial.
2. Se calcula la SCDG producida al mover cada individuo del clúster en que se encuentra hacia otro clúster.
3. Se realiza el cambio que lleva a la mayor disminución en el criterio.
4. Repetir pasos 1 y 2 hasta que ningún movimiento produzca disminuciones en el criterio.

### 15.1.2 Función de SCDG

```
load("precalculo.Rdata")
base1=base[,c(19,20,21,22,24)]
```

#### Cálculo del SCDG

```
scdg=function(base,grupo){
  k = length(unique(grupo))
  sc=0
  for(i in 1:k){
    mat=base[grupo==i,]
    centroide=apply(mat,2,mean)
    sc=sc+sum(as.matrix(dist(rbind(centroide,mat)))[1,-1]**2)
  }
  return(sc)
}
```

```
base1 = as.matrix(base1)
grupo=sample(1:3,nrow(datos),replace=T)
scdg(base1,grupo)
```

```
## [1] 3097.695
```

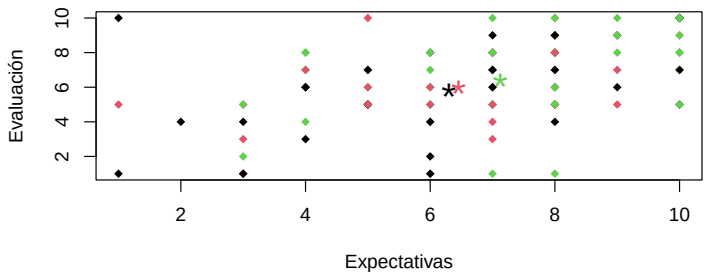
#### Obtención de los centroides

```
grupo <- sample(1:3, size = nrow(base1), replace = TRUE)
centros=matrix(nrow=3,ncol=5) # número de filas son losn grupos y las columnas número de variables
for(j in 1:5) centros[,j]=tapply(base1[,j],grupo,mean)
colnames(centros) = colnames(base1)
rownames(centros) = c(1,2,3)
centros
```

```
##   Expectativas Evaluacion EntiendeProf Materialutil CharlasCASE
## 1     6.303030   5.969697     6.696970     6.818182    3.575758
## 2     6.454545   6.121212     7.545455     7.333333    4.121212
## 3     7.121951   6.560976     7.780488     7.756098    3.780488
```

Los centroides dan muy similares en este caso por pura aleatoriedad, pero puede que no sea así

```
plot(base1[,1],base1[,2],col=grupo,pch=18,xlab="Expectativas",ylab="Evaluación")
points(centros[,1],centros[,2],col=1:3,pch="*",cex=2)
```




#### Calcular los centroides y dependiendo de la posición de estos se reasignan los grupos

```r
grupo1=c()
for(i in 1:nrow(base1)){
  disti=c()
  for(j in 1:3){
    disti[j]=dist(rbind(centros[j,],base1[i,]) )
  }
  grupo1[i]=which(disti==min(disti))
}
```

### Comparación directa

```r
head(cbind(grupo, grupo1),10)
```

```
##       grupo grupo1
##  [1,]     3      1
##  [2,]     3      3
##  [3,]     1      1
##  [4,]     3      3
##  [5,]     3      1
##  [6,]     2      3
##  [7,]     1      3
##  [8,]     2      1
##  [9,]     3      3
## [10,]     3      2
```

### Comparación para observar el cambio

```r
table(grupo,grupo1)
```

```
##      grupo1
## grupo  1  2  3
##     1 17  5 11
##     2 12  3 18
##     3 14  7 20
```

Esto nos quiere decir que los que comenzaron en la primer agrupación en el grupo 1 y terminaron en el grupo 3 en la segunda agrupación fueron 14. *Nota:* La diagonal son lo que se mantienen en el mismo grupos siempre.

```r
datos = base1
grupo=sample(1:3,nrow(datos),replace=T)
scdg1 = c()
n.it = 10
check=1
it = 1
```

```r
# 1. Guardar la configuración gráfica original y definir la matriz 2x3
par_orig <- par(no.readonly = TRUE)
par(mfrow = c(2, 3), mar = c(4, 4, 2.5, 1))

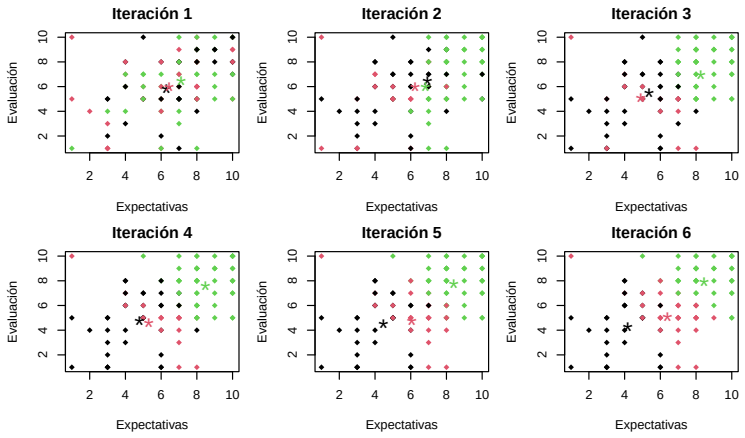
# 2. Bucle iterativo de K-medias
while(check > 0 & it < n.it){
  plot(datos[, 1], datos[, 2], col = grupo, pch = 18,
       xlab = "Expectativas", ylab = "Evaluación",
       main = paste("Iteración", it))
  points(centros[, 1], centros[, 2], col = 1:3, pch = "*", cex = 2)

  scdg1[it] = scdg(datos, grupo)
```

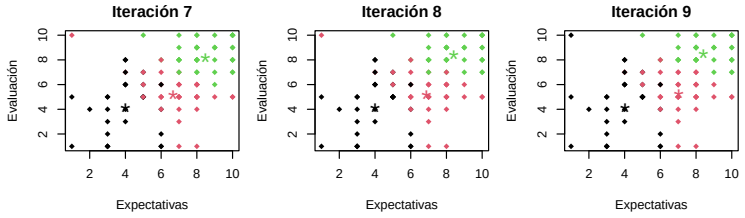
```
  for(j in 1:2) centros[, j] = tapply(datos[, j], grupo, mean)

  for(i in 1:nrow(datos)){
    disti = c()
    for(j in 1:3){
      disti[j] = dist(rbind(centros[j, ], datos[i, ]))
    }
    grupo1[i] = which(disti == min(disti))
  }

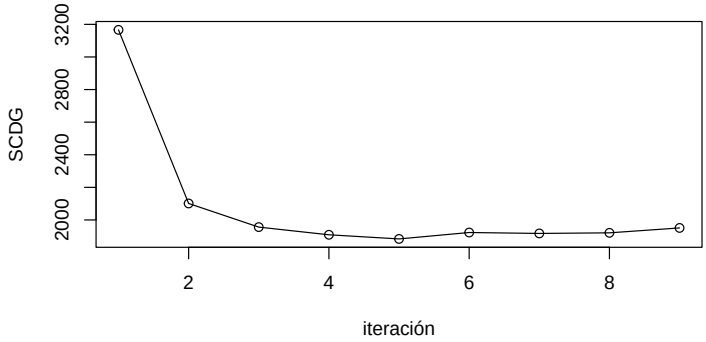
  check = sum(abs(grupo - grupo1))
  it = it + 1
  grupo = grupo1
}
```




```
# 3. Restaurar los parámetros gráficos por defecto
par(par_orig)
```




```
plot(scdg1,xlab="iteración",ylab="SCDG",type="o")
```




Conforme la cantidad de reagrupaciones aumenta, la SCDG disminuye.

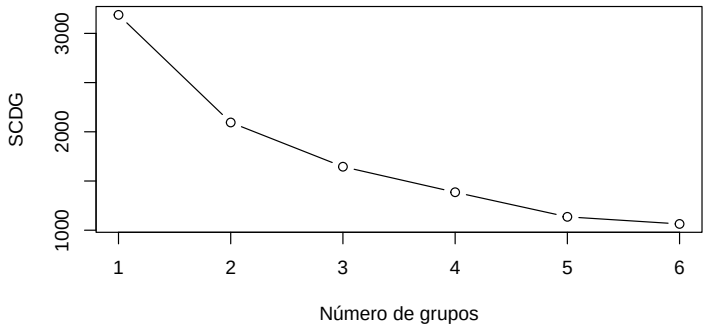
**Cálculo de la suma de cuadrados dentro para cada cantidad de grupos desde 1 hasta 6**

```
SCDG1 = c()
for (k in 1:6) SCDG1[k] = kmeans(base1,centers = k)$tot.withinss
SCDG1
```

```
## [1] 3188.131 2094.630 1645.180 1385.847 1135.892 1063.658
```

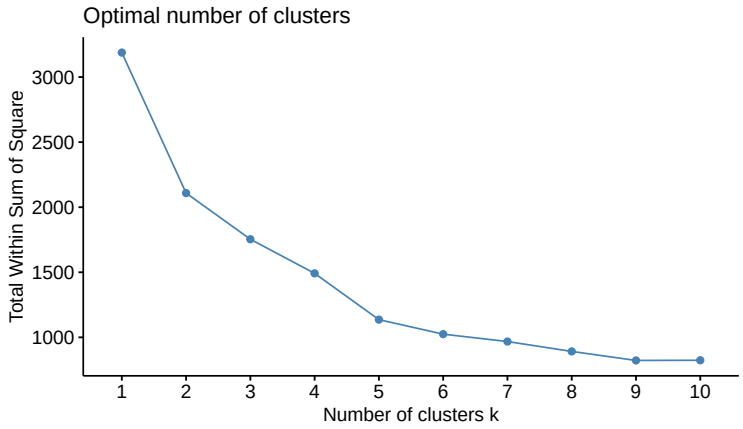
**Gráficamos el resultado anterior**

```
plot(1:6, SCDG1, type = "b", xlab = "Número de grupos",ylab = "SCDG")
```




**Forma automática de hacer todo lo anterior**

```
library(factoextra)
d = dist(base1)
fviz_nbclust(base1, kmeans, method = "wss", diss = d)
```




**Otra forma automática**

```
km=kmeans(base1,centers = 3)
```

### 15.1.3 Algoritmo 2

Se toma una partición arbitraria en k clústers que puede venir del resultado de otro método de agrupamiento.

*A continuación se le da la serie de pasos para este algoritmo*

1. Se obtiene el centroide de cada clúster.
2. Se asigna cada observación al grupo con el centroide más cercano.
3. Repetir pasos 1 y 2 hasta que no se observen movimientos de observaciones entre clústers.

### 15.1.4 Desventajas

Este método no es invariante ante cambios de escala.

Impone una estructura esférica en los datos aún cuando no se comporten de esa forma.

Las agrupaciones resultantes pueden variar dependiendo de la asignación aleatoria inicial de los centroides. Para minimizar este problema se recomienda repetir el proceso de *clustering* entre 25-50 veces y seleccionar como resultado definitivo el que tenga menor SCDG. Aun así, solo se puede garantizar la reproducibilidad de los resultados si se emplean semillas.

Presenta problemas de robustez frente a *valores atípicos*. La solución es excluirlos o recurrir a otros métodos de clustering más robustos como *K-medoides*

### 15.1.5 Mezcla de dendograma con k-medias

```
hkm = hkmeans(x = base1, hc.metric = "euclidean",hc.method = "ward.D", k = 4)
```

## 15.2 k-medoides

**Medoide:** elemento dentro de un cluster cuya distancia promedio entre él y todos los demás elementos del mismo cluster es lo menor posible. Es el elemento ubicado más hacia el centro del cluster y, por lo tanto, puede considerarse como el más representativo. El hecho de utilizar medoides en lugar de centroides hace de K-medoids un método más robusto que K-medias, viéndose menos afectado por valores atípicos o ruido. A modo de idea intuitiva puede considerarse como la analogía entre media y mediana.

*A continuación se le presenta un ejemplo con la diferencia entre centroide y medoide*

Tenemos los siguientes datos

| Observación | Peso (kg) | Altura (cm) |
|---|---|---|
| $P_1$ | 60 | 160 |
| $P_2$ | 65 | 165 |
| $P_3$ | 70 | 190 |
| **Centroide (g)** | **65.00** | **171.67** |

**Cálculo del centroide**

Coordenada del Peso:

$$\bar{x}_{\text{peso}} = \frac{1}{3}\sum_{i=1}^{3} x_{i1} = \frac{60+65+70}{3} = \frac{195}{3} = \mathbf{65.00}\ \text{kg}$$

Coordenada de la Altura:

$$\bar{x}_{\text{altura}} = \frac{1}{3}\sum_{i=1}^{3} x_{i2} = \frac{160+165+190}{3} = \frac{515}{3} \approx \mathbf{171.67}\ \text{cm}$$

### 15.2.1 Cálculo medoide

Teniendo claro esto también podemos calcular el mediode, de la siguiente manera:

Primero calculamos las distancias

```
pesos = c(60, 65, 70)
alturas = c(160, 168, 190)

datos = cbind(pesos, alturas)

d = dist(datos, method = "manhattan")
m = as.matrix(d)
m
```

```
##    1  2  3
## 1  0 13 40
## 2 13  0 27
## 3 40 27  0
```

Luego sumamos las filas para ver cuál punto esta más cerca de los demás:

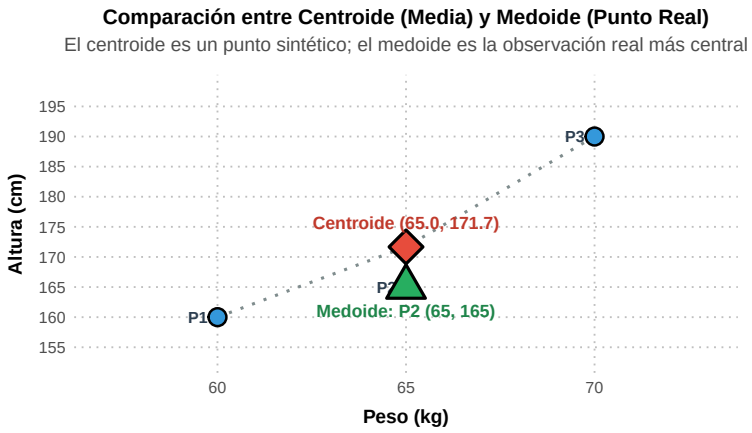
```
apply(m, 2, \(x) mean(x[x != 0])) # Este código es para no incluir a el cero en el promedio
```

```
##    1    2    3
## 26.5 20.0 33.5
```

En este caso el medoide es el punto 2

**Otra forma**

```
matriz_distancias <- as.matrix(dist(datos[, c("pesos", "alturas")]))
indice_medoide <- which.min(rowSums(matriz_distancias))
medoide <- datos[indice_medoide, ]
```




## 15.3 Algoritmo PAM (partitioning around medoids)

Se toman aleatoriamente $k$ observaciones como medoides iniciales. Previamente se obtiene la matriz de distancias entre todas las observaciones.

*A continuación se le da la serie de pasos para este algoritmo*

1. Se asigna cada observación al grupo representado por el medoide más cercano.
2. Para cada uno de los clusters creados, comprobar si seleccionando otra observación como medoide se consigue reducir la distancia promedio del cluster, si esto ocurre, seleccionar la observación que consigue una mayor reducción como nuevo *medoide*.
3. Si al menos un medoide ha cambiado en el paso 2, volver al paso 1, de lo contrario, se termina el proceso.

### 15.3.1 Observaciones sobre k-medoides

Por lo general, este método se utiliza cuando se conoce o se sospecha de la presencia de valores atípicos.

Es recomendable utilizar como medida de similitud la distancia de Manhattan, ya que es menos sensible a valores atípicos que la euclídea.

Para conjuntos de datos grandes necesita muchos recursos computacionales. En tal situación se recomienda aplicar el método CLARA.

### 15.3.2 CLARA (clustering large applications)

Combina la idea de k-medoides con remuestreo para aplicarlo a grandes volúmenes de datos.

Se selecciona una muestra aleatoria de un tamaño determinado y se aplica el algoritmo de PAM para encontrar los clústers óptimos acorde a esa muestra.

Utilizando esos medoides se agrupan las observaciones de toda la base de datos.

Se calcula la suma de las distancias entre cada observación de la base de datos y su correspondiente medoide.

Se repite este proceso un número predeterminado de veces con el objetivo de reducir el sesgo de muestreo.

Se seleccionan como clústers finales los obtenidos con aquellos medoides que han conseguido la menor suma total de distancias.

### 15.3.3 Formas automáticas

```
library(cluster)
kmedoids=pam(base1,3)
```

*A continuación se usa su variante CLARA*

```
clara1=clara(base1,k=3,metric="euclidean",stand=T,samples=2,pamLike = T)
```

# 16 Creación de grupos o clusteres

Se puede crear una variable categórica, que es la asignación de a cuál grupo pertenece la observación según el tipo de distancia hechos en los dendogramas o diferentes análisis.

**Con las distancias**

```
# cada uno de los c se obtuvieron en los dendogramas de arriba con su respectivo corte
clust.single = cutree(cs, k=3)
clust.complete = cutree(c.com, k=3)
clust.av = cutree(c.av, k=3)
clust.ward = cutree(c.w, k=3)
```

**Con difereres análisis**

```
clust.km=km$cluster
clust.kmedoid=kmedoids$cluster
clust.clara  = clara1$cluster
clust.hkm  = hkm$cluster
```

**Comparación de los diferentes métodos**

```
table(clust.km,clust.kmedoid)
```

```
##          clust.kmedoid
## clust.km  1  2  3
##        1 28  0  0
##        2  0 42  5
##        3  2  0 30
```

*Nota: Recordar hacer el boxplot o algún gráfico que me ayuda a observar las diferencias entre grupos y ver que claramente estos grupos tienen sentido. Ya que lo que se quiere observar es que los grupos sean lo mas diferentes posibles, pero dentro de estos no sean muy variables*

```
library(dendextend)
corden=cor.dendlist(dendlist("cs"=as.dendrogram(cs),"csM"=as.dendrogram(csM),"csMan"=as.dendrogram(csMa
round(corden,1)
```

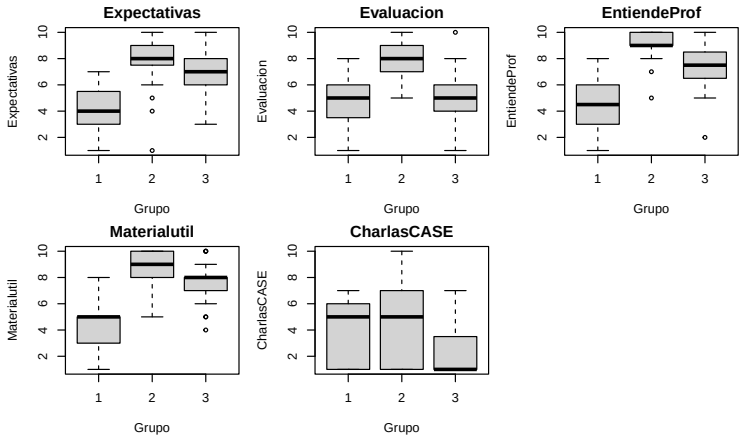
### 16.0.1 Comparación de grupos con variables continuas

```
par_orig = par(no.readonly = TRUE)
par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))

p = ncol(base1)
nombres = colnames(base1)

# 2. Bucle de boxplots
for(i in 1:p){
  boxplot(base1[, i] ~ clust.km,
          xlab = "Grupo",
```

```
          ylab = nombres[i],
          main = nombres[i])
}
par(par_orig)
```



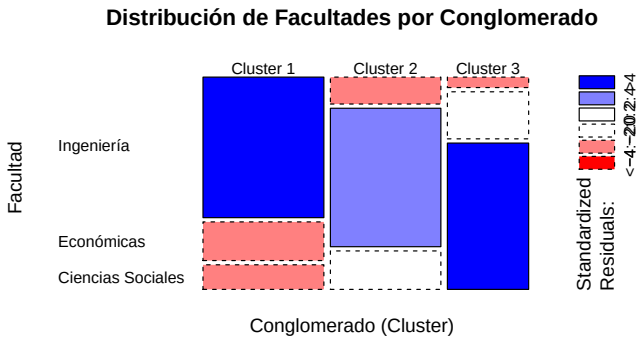

La idea de hacer estos boxplots es observar diferencias entre grupos y dentro de los mismos poca variabilidad

### 16.0.2 Comparación de grupos con variables categóricas

A continuación se tienen los siguentes datos

```
##            Facultad
## Cluster     Ingeniería Económicas Ciencias Sociales
##   Cluster 1         40         11                 7
##   Cluster 2          7         36                10
##   Cluster 3          2          9                28
```

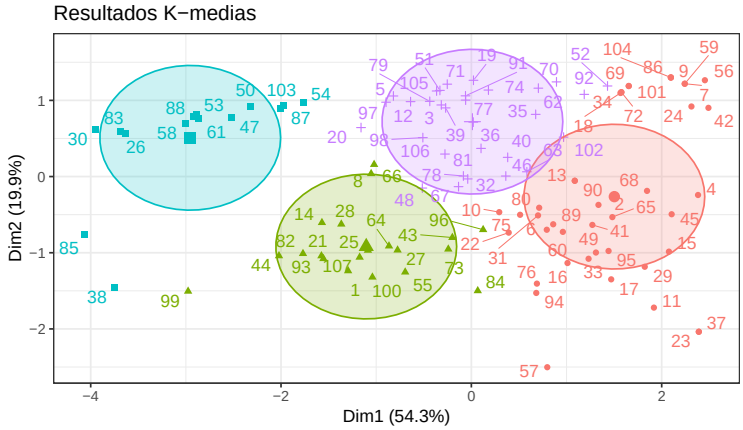
```
#par(mar = c(4, 4, 3, 2))
mosaicplot(
  tabla_contingencia,
  shade = TRUE,
  main  = "Distribución de Facultades por Conglomerado",
  xlab  = "Conglomerado (Cluster)",
  ylab  = "Facultad",
  las   = 1,
  cex.axis = 0.85
)
```




### 16.0.3 K-medias con PCA, para graficar

La cantidad de grupos la personalizamos dentro del cálculo de `kmeans` que se le da a la función.

```
fviz_cluster(kmeans(base1,centers=4), base1, show.clust.cent = TRUE,
             ellipse.type = "euclid", repel = TRUE) +
  labs(title = "Resultados K-medias") +
  theme_bw() + theme(legend.position = "none")
```

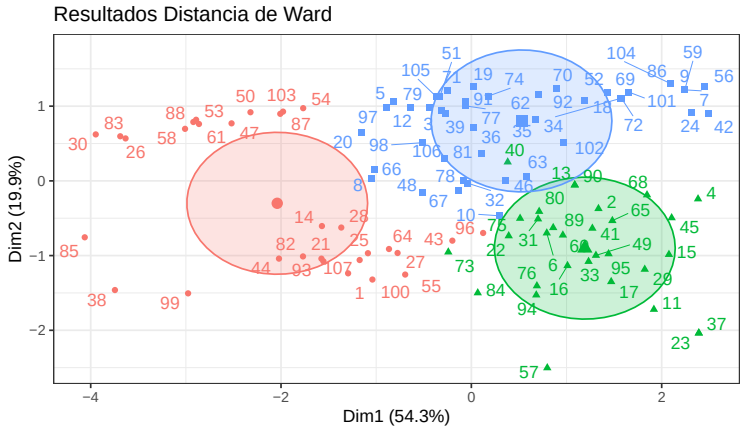



La idea es que todos los puntos se encuentren dentro de los elipses.

### 16.0.4 Combinación de dendograma con gráfico de clúster

```
c.w2 = eclust(base1, k=3,"hclust", hc_metric = "euclidean",hc_method = "ward.D",nboot = 2)
```

```
fviz_cluster(c.w2, base1,show.clust.cent = TRUE,
             ellipse.type = "euclid", repel = TRUE) +
  labs(title = "Resultados Distancia de Ward") +
  theme_bw() +
  theme(legend.position = "none")
```




## 16.1 Métodos difusos

La función `fanny` (fuzzy analysis) de la librería `cluster` permite aplicar el algoritmo c-means clustering (FCM). El parámetro `diss=T` se usa cuando se trabaja a partir de una matriz de distancias, de lo contrario se usa `diss=F` cuando se trabaja con los datos originales. Se debe indicar en `metric` la distancia entre individuos, en `k` el número de grupos y en `stand` si se quiere estandarizar los datos.

*A continuación se le presenta este método con la distancia euclideana para 3 grupos*

```
fcm=fanny(base1,k=3,diss=F,metric="euclidean",stand = F)
# Si se quiere usar una matriz de distancias específicas se tiene que colocar diss = T
```

```
head(fcm$membership,5)
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.3333334 0.3333333 0.3333333
## [2,] 0.3333333 0.3333334 0.3333333
## [3,] 0.3333333 0.3333333 0.3333333
## [4,] 0.3333333 0.3333334 0.3333333
## [5,] 0.3333334 0.3333333 0.3333333
```

La idea es que las probabilidades sean lo mas distintas entre sí, de modo que haya una muy grande y las demás muy pequeñas. En la tabla anterior no sucede lo que se quiere con este análisis. Cuándo muchos puntos tienen las probabilidades muy similares, como sucedió en el caso anterior se dice que es una base de datos difusa.

Si se usa la distancia de Manhattan en ocasiones el algoritmo de este método no converge por lo que se tiene que usar la siguiente función:

```
fcm=fanny(base1,k=3,diss=F,metric="manhattan",stand = F,maxit = 10000)
```

```
head(fcm$membership,5)
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.4179941 0.2909970 0.2910089
## [2,] 0.2196863 0.3901651 0.3901486
## [3,] 0.3637407 0.3181259 0.3181334
## [4,] 0.2282669 0.3858769 0.3858563
## [5,] 0.4027272 0.2986306 0.2986422
```

En este caso la situación mejoró ya que se eliminaron un poco los empates.

### 16.1.1 Código para sacar la distancia entre la mayor y menor probabilidad de cada observación

```
U = fcm$membership
n = nrow(U)
dif_top2 = c()

for (i in 1:n) {
  fila = U[i, ]
  top2 = sort(fila, decreasing = TRUE)[1:2]
  dif_top2[i] = top2[1] - top2[2]
}
idx_menor = which(dif_top2 < 0.03)
idx_mayor = which(dif_top2 >= 0.03)  # Casos bien definidos

# Conteos
table(dif_top2 > 0.03) # Condición de cuales tienen una diferencia mayor al umbral
```

```
##
## FALSE  TRUE
##    68    39
```

### 16.1.2 ¿Que tán difusos están los datos?

Note que hay individuos que tienen probabilidades de pertenencia muy similares en varios grupos. Esto es una indicación de que hay mucho puntos frontera (difusos). Para medir el nivel de difusión, existe el índice de Dunn, los valores normalizados próximos a 0 indican que la estructura tiene un alto nivel difuso y valores próximos a 1 lo contrario. Para obtener esto use `fcm$coef`.

```
fcm$coeff
```

```
## dunn_coeff normalized
##  0.3426727  0.0140091
```

## 16.2 Obtención del número de grupos óptimo mediante el método de la silueta promedio

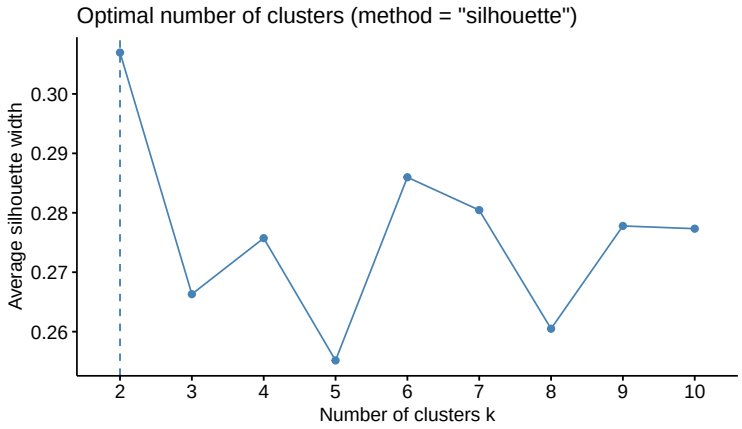
Para cada observación $i$ se calcula el coeficiente silueta de la siguiente forma:

- Calcular el promedio de las distancias entre la observación $i$ y el resto de observaciones que pertenecen al mismo cluster. Este valor se llama $a_i$. Cuanto menor sea $a_i$, mejor ha sido la asignación de $i$ a su cluster.
- Calcular la distancia promedio entre la observación $i$ y el resto de clusters. Entendiendo por distancia promedio entre $i$ y un determinado clúster $C$ como la media de las distancias entre $i$ y las observaciones del clúster $C$.
- Llame $b_i$ a la menor de las distancias promedio entre $i$ y el resto de los clústers, es decir, la distancia al cluster más próximo.

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

*A continuación se le presenta el código en r de este proceso*

```
fviz_nbclust(base1, kmeans, method = "silhouette", diss = d)
```




### 16.2.1 Cluster Jerárquico con Silueta

```
length(cutree(c.com, 2))  # Observaciones en el árbol c.com
```

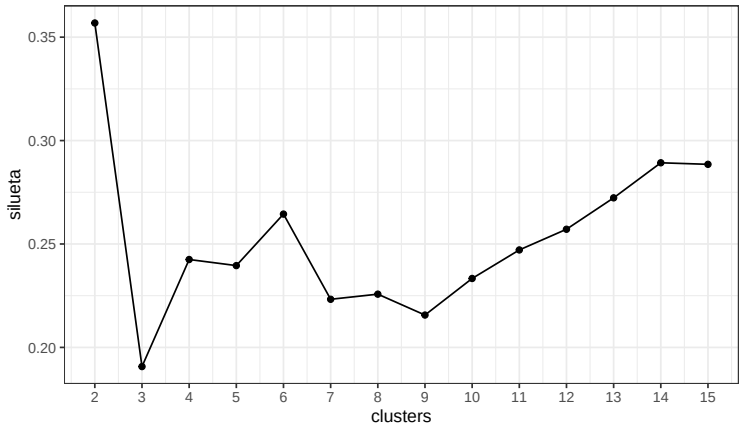
```
## [1] 107
```

```
nrow(base1)               # Observaciones en base1
```

```
## [1] 107
```

```
sil.k=c()
n.clust=2:15
for(i in 1:length(n.clust)){
  s=silhouette(cutree(c.com,n.clust[i]),get_dist(x = base1, method = "euclidean"))
  sil.k[i]=summary(s)[[4]]
}

data.frame(clusters = 2:15, silueta = sil.k) %>%
  ggplot(aes(x = clusters, y = silueta)) +
    geom_line() +
    geom_point() +
    scale_x_continuous(breaks = 2:15) +
    theme_bw()
```



## 16.3 Estadístico GAP

Este estadístico compara, para diferentes valores de $k$, la varianza total *intra-cluster* observada frente al valor esperado acorde a una distribución uniforme de referencia.

La estimación del número óptimo de clusters es el valor $k$ con el que se consigue maximizar el estadístico gap, es decir, encuentra el valor de $k$ con el que se consigue una estructura de clusters lo más alejada posible de una distribución uniforme aleatoria.

Este método puede aplicarse a cualquier tipo de *clustering*.

### 16.3.1 Construcción del estadístico WAP

Hacer clustering de los datos para un rango de valores de k y para cada uno calcular la variancia total intracluster ($W_k$). Es decir que tan separados están los clusters.

Simular B conjuntos de datos de referencia, todos con una distribución uniforme. Aplicar clustering a cada uno de los conjuntos con el mismo rango de valores k empleado en los datos originales, calculando en cada caso la varianza total intra-cluster ($W_{kb}$). Se recomienda $B = 500$.

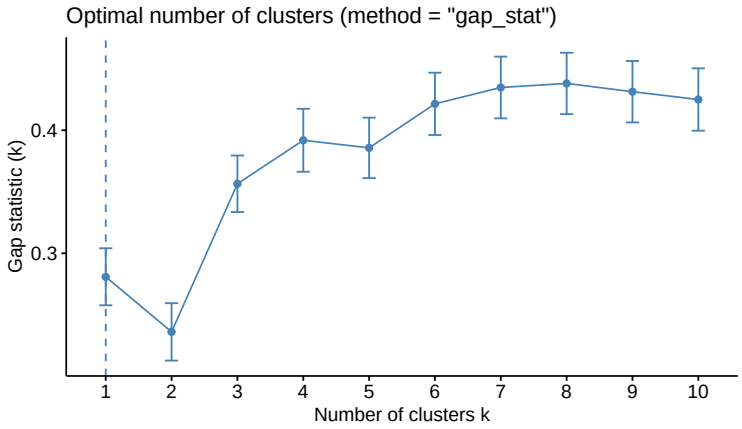
Calcular el estadístico gap para cada valor de k:

$$\begin{aligned} gap(k) &= \frac{1}{B}\sum_{b=1}^{B}\log(W_{kb}) - \log(W_k) \\ &= \frac{1}{B}\sum_{b=1}^{B}\log\left(\frac{W_{kb}}{W_k}\right) \end{aligned}$$

*Nota*: Si $\frac{W_{kb}}{W_k} \to 1$, el resultado tiende a cero.

*A continuación se le presenta el código en r para realizar el cálculo de este estadístico*

```
fviz_nbclust(base1, kmeans, method = "gap", diss = d)
```

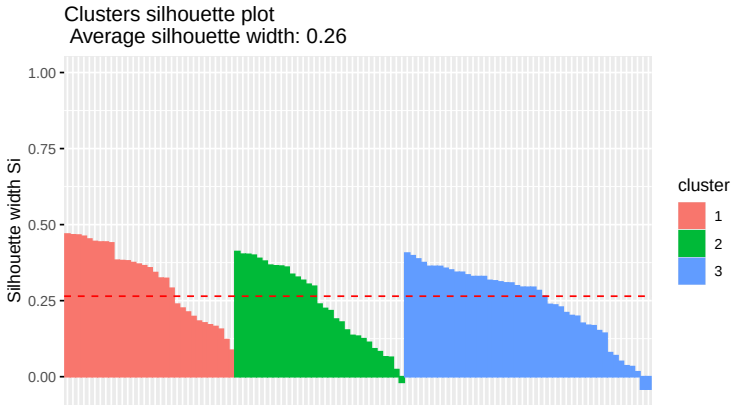



## 16.4 Validación

El uso combinado de las funciones `eclust` y `fviz_silhouette` de la librería `factoextra` permiten obtener los coeficientes de silueta de forma sencilla. La función `eclust`, gracias a su argumento `FUNcluster`, facilita el uso de múltiples algoritmos de clustering mediante una misma función (internamente llama a las funciones kmeans, hclust, pam, clara, etc). Use las siguientes instrucciones:

```
km1 = eclust(base1, FUNcluster = "kmeans", k = 3, hc_metric = "euclidean", nstart = 50, graph = F)
fviz_silhouette(sil.obj = km1, print.summary = TRUE)
```

```
##   cluster size ave.sil.width
## 1       1   31          0.32
## 2       2   31          0.24
## 3       3   45          0.24
```




La idea con este gráfico es que todos los valores siempre se vean positivos, en este caso el cluster tres podemos ver que tienen algunos puntos con una muy baja silueta, esto nos indica que algunos puntos están como en la frontera, para solucionar ese problema usamos los métodos difusos que vimos anteriormente y que se encuentran en páginas anteriores.

## 16.5 Indice Dunn

### 16.5.1 Índice Dunn

Para cada clúster, se calcula la distancia intracluster como el máximo de las distancias entre todos los individuos de ese clúster.

$$\Delta_i = \max_{x,y \in C_i} (x, y)$$

También puede usar como distancia intraclusters la distancia media entre todos los pares del clúster, o la distancia media de todos los puntos a su centroide.

Para cada par de clústers, se calcula una medida de distancia entre clústers ($D_{ij}$).

El índice Dunn se calcula como:

$$D = \frac{\min_{i \neq j} D_{ij}}{\max_k \Delta_k}$$

Si la estructura contiene *clusters* compactos y bien separados, el numerador es grande y el denominador pequeño, dando lugar a valores altos de D.

El objetivo es maximizar el índice *Dunn*.

Esta forma de evaluar la calidad del *clustering* tiene un inconveniente. Si todos los *clusters* tienen un comportamiento ideal excepto uno, cuya calidad es baja, dado que el denominador emplea el máximo en lugar de la media, el índice estará totalmente influenciado por este *clúster* enmascarando al resto. Es importante tener en cuenta que se trata de un indicador de tipo "el peor de los casos".

*A continuación se le presenta el código en r para obtener el indice*

```r
#install.packages("fpc")
library(fpc)
km_indices = cluster.stats(d,clust.km)
km_indices$dunn
```

```
## [1] 0.1344332
```

*Nota:* Lo que se quiere siempre es buscar un índice de Dunn lo más bajo posible.

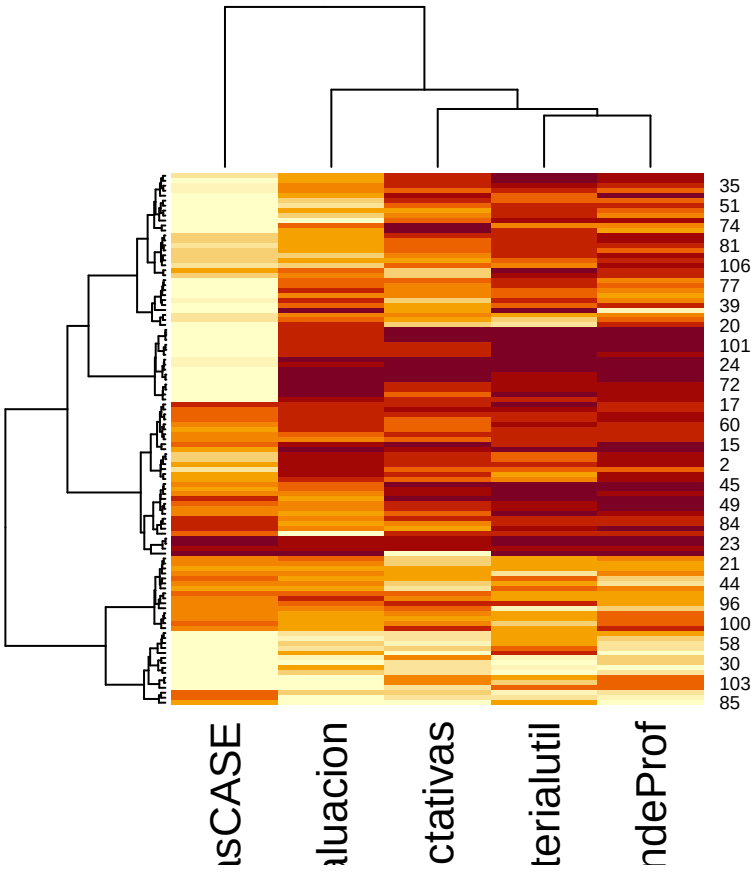
## 16.6 Mapas de calor (heatmaps)

Los mapas de calor son el resultado obtenido al representar una matriz de valores en la que, en lugar de números, se muestra un gradiente de color proporcional al valor de cada variable en cada posición.

La combinación de un dendrograma con un heatmap permite ordenar por semejanza las filas y/o columnas de la matriz, a la vez que se muestra con un código de colores el valor de las variables.

Se consigue así representar más información que con un simple dendrograma y se facilita la identificación visual de posibles patrones característicos de cada cluster.

```r
heatmap(base1, scale = "none",
        distfun = function(x){dist(x, method = "euclidean")},
        hclustfun = function(x){hclust(x, method = "ward.D")},
        cexRow = 0.7)
```




Observe que en el anterior gráfico se puede ver que el dendograma del eje vertical divide dos grupos, por esta razón es que la parte de abajo es más claro que las partes de arriba. Lo mismo ocurre con el dendograma de la parte superior en donde igualmente se divide en dos grupos y por ello la parte de la izquierda del mapa de calor es más claro que las parte que siguen a la derecha.

*Nota:* Este gráfico está hecho para dendograma si el análisis que está ganando es el de k-medias o el de k-mediodes, esta figura no aplica.

# 17 Escalamiento multidimensional (MDS)

Se trata de agrupar los individuos según las distancias de sus observaciones en las distintas variables (entre más cerca se encuentren dos individuos más parecidos son).

En la construcción de las nuevas coordenadas basadas en la distancia no existe una variable respuesta. Todas las variables funcionan al mismo nivel y se construyen nuevas variables a partir de ese conjunto de variables disponibles.

## 17.1 Escalamiento multidimensional métrico (MMDS)

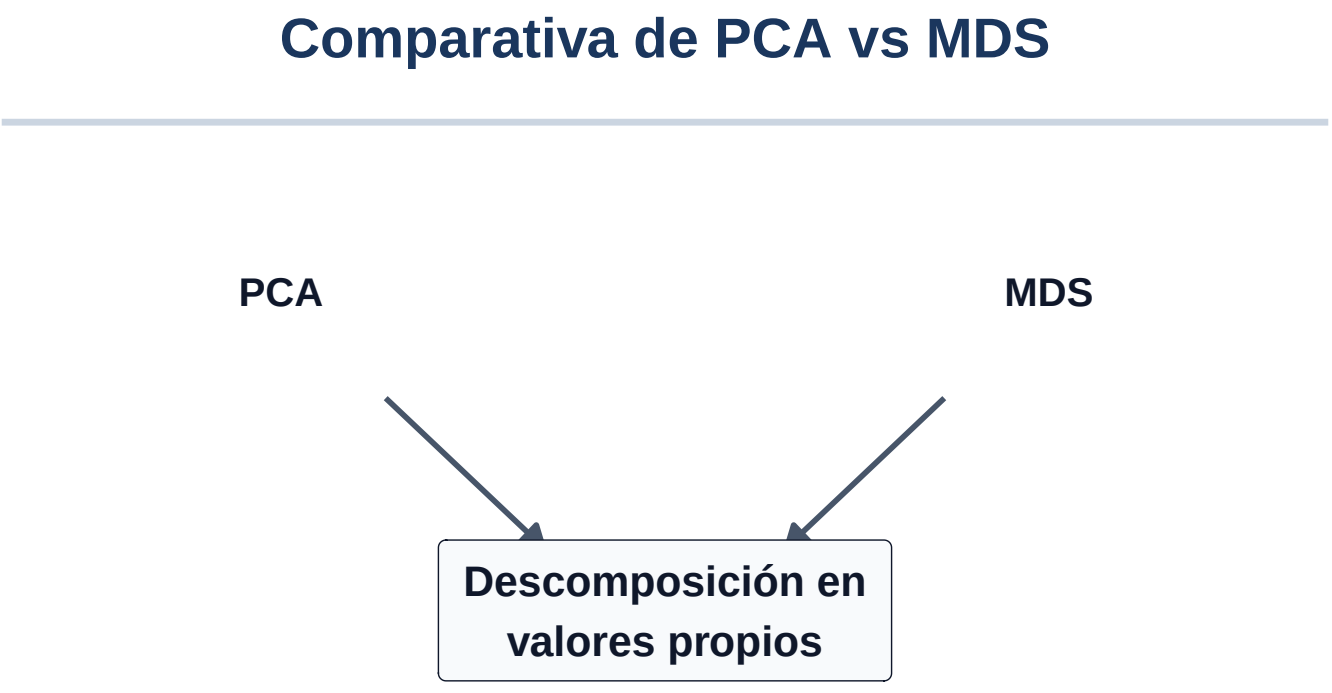
El escalamiento multidimensional métrico es también conocido como escalamiento multidimensional clásico (CMDS) o análisis de coordenadas principales (PCoA).

El escalamiento multidimensional clásico se puede hacer con cualquier distancia que el investigador considere pertinente: euclídea, manhattan, mahalanobis, etc.

### 17.1.1 Datos Curiosos de MMDS

Si se realiza el MMDS con la distancia euclídea se obtiene el mismo resultado que realizar un análisis de componentes principales.

Por ende, agrupar minimizando las distancias lineales entre individuos (MMDS) es lo mismo que agrupar maximizando las correlaciones lineales entre esos mismos individuos (PCA).



1. **Coordenadas** para una representación gráfica más simple.
2. **Porcentaje de variación explicada** proporción de los valores propios.
3. **Acomodo de los componentes principales** / las coordenadas en orden de importancia.

A partir de la matriz de datos $\mathbf{X}$ se obtiene una matriz de distancias $\mathbf{D}$ de dimensión $n \times n$. El MMDS parte de $\mathbf{D}$ y trata de encontrar $\mathbf{X}$.

Para comprender la mecánica se asume que $\mathbf{X}$ es conocida y se obtiene el producto $\mathbf{B} = \mathbf{X}\mathbf{X}^\top$. Entonces cada elemento de $\mathbf{B}$ se puede expresar como:

$$b_{ij} = \sum_{k=1}^{q} x_{ik} x_{jk}$$

De esta forma la distancia Euclídea al cuadrado se puede expresar como:

$$d_{ij}^2 = b_{ii} + b_{jj} - 2b_{ij}$$

Se impone la restricción de que los datos de cada variable estén centrados en cero, por lo que $\sum_{i=1}^{n} x_{ik} = 0$. De esta forma todos los elementos de $\mathbf{B}$ son tales que la suma de cualquier fila (o columna) es cero: $\sum_{j=1}^{n} b_{ij} = 0$. Se obtienen las siguientes relaciones:

$$\sum_{i=1}^{n} d_{ij}^2 = T + nb_{jj} \qquad \sum_{j=1}^{n} d_{ij}^2 = T + nb_{ii} \qquad \sum_{i=1}^{n}\sum_{j=1}^{n} d_{ij}^2 = 2nT$$

donde $T = \mathrm{tr}(\mathbf{B}) = \sum_{i=1}^{n} b_{ii}$. De esta forma:

$$b_{ij} = -\frac{1}{2}\left(d_{ij}^2 - d_{i\cdot}^2 - d_{\cdot j}^2 + d_{\cdot\cdot}^2\right)$$

donde las medias marginales y global de las distancias al cuadrado se definen como:

$$d_{i\cdot}^2 = \frac{1}{n}\sum_{j=1}^{n} d_{ij}^2 \qquad d_{\cdot j}^2 = \frac{1}{n}\sum_{i=1}^{n} d_{ij}^2 \qquad d_{\cdot\cdot}^2 = \frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n} d_{ij}^2$$

Ahora se hace la descomposición espectral de $\mathbf{B}$:

$$\mathbf{B} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$$

La matriz $\mathbf{\Lambda}$ es diagonal con los $n$ valores propios de $\mathbf{B}$ en la diagonal. Si el rango de $\mathbf{B}$ es $q$, entonces los últimos $n - q$ valores propios son cero por lo que se pueden eliminar y escribir $\mathbf{B}$ en función solamente de los primeros $q$ vectores y valores propios. Finalmente se puede escribir la matriz $\mathbf{X}$ como:

$$\mathbf{X} = \mathbf{V}_1\mathbf{\Lambda}_1^{1/2}$$

Donde $\mathbf{V}_1$ es la matriz que contiene solo los primeros $q$ vectores propios, y $\mathbf{\Lambda}_1$ es la matriz diagonal con solo los primeros $q$ valores propios. La mejor representación de dimensión $k$ está dada por los $k$ vectores propios de $\mathbf{B}$, asociados a los $k$ valores propios más grandes.

*A continuación todo los ejemplos de este de tema se basaran en los siguientes datos*

```r
base = cbind(c(3,5,6,1,4,2,0,0,7,2),
             c(4,1,2,1,7,2,4,6,6,1),
             c(4,1,0,1,3,5,1,4,5,4),
             c(6,7,2,0,6,1,1,3,1,3),
             c(1,3,6,3,2,0,1,5,4,1))
```

```r
d = dist(base)
```

### 17.1.2 Escalamiento Multidimensional en r

'cmds<-cmdscale(d,k=5,eig=TRUE,x.ret=TRUE)', donde

- d <- es la matriz de distancias
- k <- indica el número de dimensiones requeridas
- eig <- indica si debería devolver los valores propios
- X.ret <- devuelve parte de la matriz B (La matriz B se obtiene multiplicando (-1/2)*cmds$x)

Tienen que agregarse todos estos argumentos porque de lo contrario solo da los primeros vectores propios.

```r
cmds = cmdscale(d,k=5,eig=TRUE,x.ret=TRUE) # 5 dimensiones, # d tiene que seer un objeto distancia
```

```r
cmds2 = cmdscale(d,k=2,eig=TRUE,x.ret=TRUE) # 2 dimensiones
```

### 17.1.3 Reconstrucción por la matriz B

Los elementos de la matriz B se pueden obtener de acuerdo a las fórmulas vistas en la presentación en clase o también utilizando la siguiente fórmula: $B = -\frac{1}{2}CD^2C$. Donde $D^2$ es la matriz de distancias al cuadrado y $C$ está definida como $C = I - \frac{1}{n}J_n$. Aquí $I$ es la matriz identidad de dimensión $n \times n$ y $J_n$ es una matriz de unos, de dimensión $n \times n$.

```r
n = dim(base)[1]
D = as.matrix(dist(base,diag = T,upper = T))
Jn = matrix(1,n,n)
C = diag(n)-(1/n)*Jn

B = -0.5*C%*%(D^2)%*%C
```

## 17.2 Bondad de ajuste (Mardia et al. 1979)

La idoneidad de las nuevas coordenadas resultantes del escalamiento multidimensional, se puede juzgar según la proporción de valores propios representados por dichas coordenadas.

$$P_k^{(1)} = \frac{\sum_{i=1}^{k} |\lambda_i|}{\sum_{i=1}^{n} |\lambda_i|} \qquad P_k^{(2)} = \frac{\sum_{i=1}^{k} \lambda_i^2}{\sum_{i=1}^{n} \lambda_i^2}$$

Donde entre más cercano esté a uno, mejor es la representación de los datos en las nuevas coordenadas.

*Nota:* Note que la cantidad de valores propios es igual al número de observaciones.

Sin embargo, dicha proporción nunca es igual a uno puesto que lo que se quiere es disminuir dimensiones, para poder tener una representación más sencilla de los datos.

$$k < q$$

donde:

- $q$: es el número de dimensiones originales.

- $k$: es el número de dimensiones escogidas después de aplicar el escalonamiento multidimensional.

Se recomiendan valores mayores que *0,8* para ambos criterios.

### 17.2.1 Obtención de valores propios (Eigen values)

```
round(cmds$eig,2)
```

```
##  [1] 75.19 58.81 49.61 30.43 10.37  0.00  0.00  0.00  0.00  0.00
```

### 17.2.2 Aplicación de la bondad de ajuste

**Primer Criterio**

Si $k = 2$, entonces el calculo es el siguiente:

```
round(sum(abs(cmds$eig[1:2]))/sum(abs(cmds$eig))*100,2)
```

```
## [1] 59.71
```

Si $k = 3$

```
round(sum(abs(cmds$eig[1:3]))/sum(abs(cmds$eig))*100,2)
```

```
## [1] 81.82
```

Se necesitarian al menos 3 dimensiones para alcanzar más de un 80%. Por lo que en este caso obtener una representación con dos dimensiones no es muy buena.

**Segundo Criterio**

Si $k = 2$, entonces el calculo es el siguiente:

```
round(sum(cmds$eig[1:2]^2)/sum(cmds$eig^2)*100,2)
```

```
## [1] 72.28
```

el ajuste es mejor que con el otro criterio

Si $k = 3$

```
round(sum(cmds$eig[1:3]^2)/sum(cmds$eig^2)*100,2)
```

```
## [1] 91.8
```

NO hay ninguno mejor que otro, lo mejor es tener un equilibrio entre los dos criterios.

## 17.3 Falta de ajuste (stress)

Es una medida de la falta de ajuste que consiste en la diferencia de las distancias observadas y las distancias predichas, estandarizadas por las distancias observadas al cuadrado para obtener valores entre 0 y 1.

Me dice que tan bien puedo reproducir los datos

$$stress = \sqrt{\frac{\sum(d_{ij} - \hat{d}_{ij})^2}{\sum d_{ij}^2}}$$

Donde entre más cercano esté a cero, mejor es la representación de los datos en las nuevas coordenadas.

Al evaluar bondad de ajuste del MDS, para este propósito Kruskal en 1964, propuso la siguiente regla de dedo:

| Stress | Bondad de ajuste |
|---|---|
| 0.000 | perfecto |
| 0.025 | excelente |
| 0.050 | bueno |
| 0.100 | aceptable |
| 0.200 | pobre |

*Notas:*

- Recuerde que puede usar cualquier distancia que sea pertinente para los datos.
- El *stresspuede* usarse también para calificar la falta de ajuste de un escalamiento multidimensional no métrico.

### 17.3.1 Aplicación del strees

Para este caso el mayor número de dimensiones puede ser $k = 5$, debido a la cantidad de filas que tienen nuestros datos (cantidad de sujetos)

Para $k = 2$

```
k = 2
coor2=cmds$points[,1:k]
d2=as.matrix(dist(coor2))
d=as.matrix(d)
sqrt(sum((d-d2)^2)/sum(d^2)) # d es la matriz de distancias observadas
```

```
## [1] 0.3371175
```

Para $k = 2$

```
k = 4
coor2=cmds$points[,1:k]
d2=as.matrix(dist(coor2))
d=as.matrix(d)
sqrt(sum((d-d2)^2)/sum(d^2))
```

```
## [1] 0.05019229
```

Con el resultado anterior podemos observar que con cuatro dimensiones alcanza un ajuste bueno, mas no excelente.

### 17.3.2 Comparación de valores de r de coordernadas con los valores propios

```
eB = eigen(B)
```

```
cmds$points[,1:2]
```

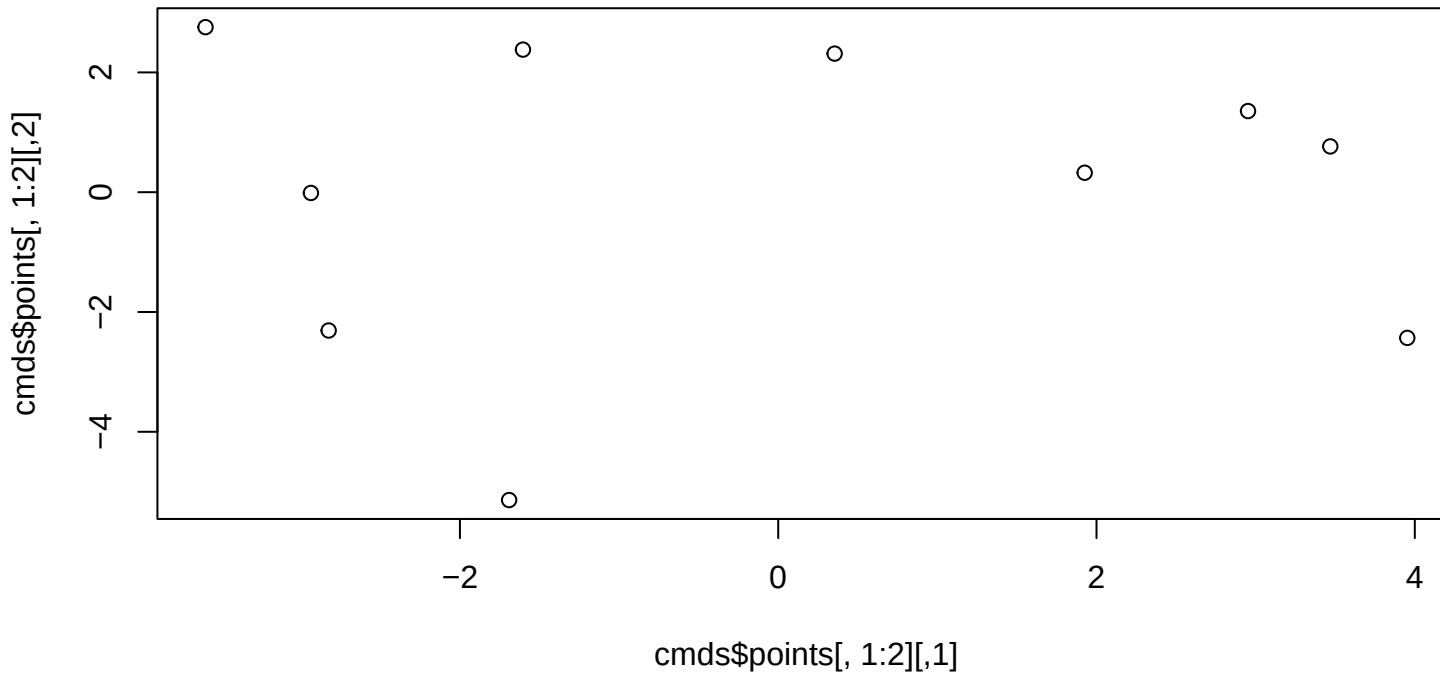
```
##              [,1]        [,2]
##  [1,] -1.6038325  2.38060903
##  [2,] -2.8246377 -2.30937202
##  [3,] -1.6908272 -5.13970089
##  [4,]  3.9527719 -2.43233961
##  [5,] -3.5984894  2.75538195
##  [6,]  2.9520356  1.35475175
##  [7,]  3.4689928  0.76411068
##  [8,]  0.3545235  2.31408566
```

```
##  [9,] -2.9362323 -0.01279597
## [10,]  1.9256952  0.32526941
```

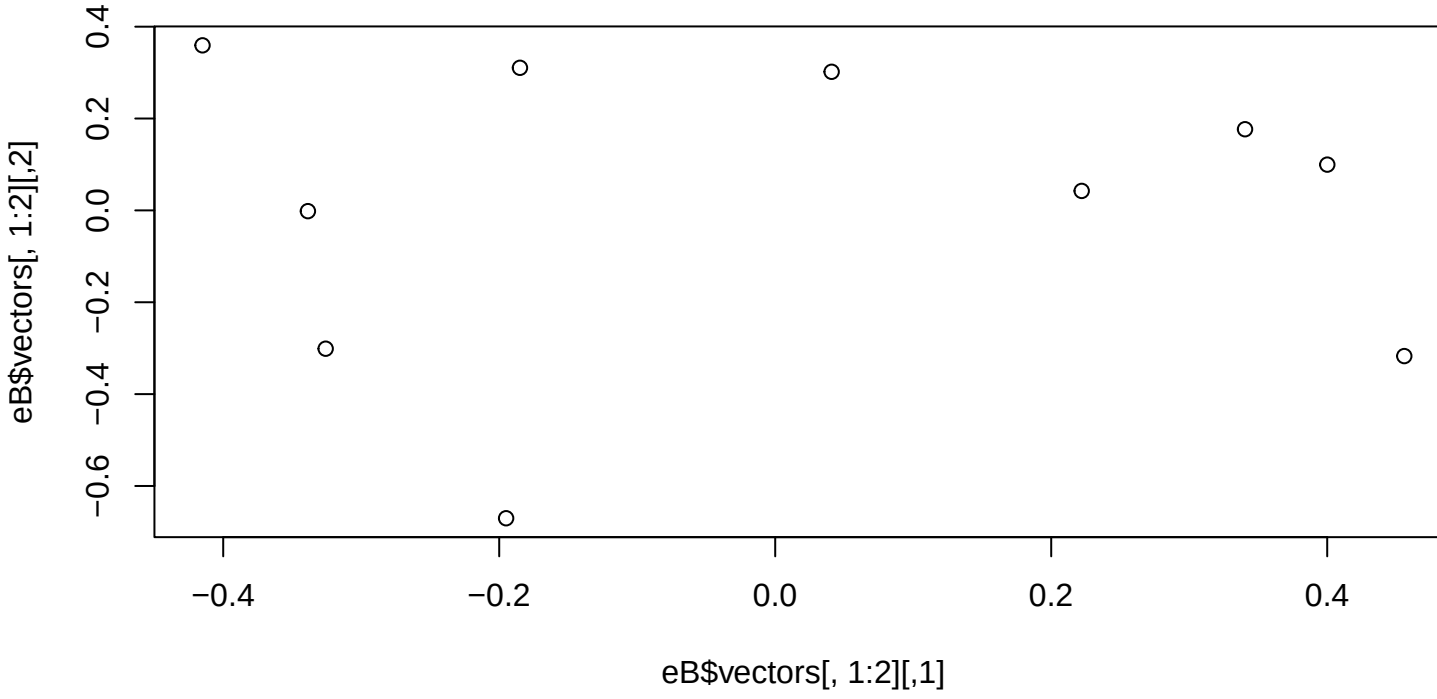
```
eB$vectors[,1:2]
```

```
##              [,1]         [,2]
##  [1,] -0.18496398  0.310440748
##  [2,] -0.32575486 -0.301151163
##  [3,] -0.19499676 -0.670237142
##  [4,]  0.45585835 -0.317186619
##  [5,] -0.41500028  0.359312607
##  [6,]  0.34044719  0.176664938
##  [7,]  0.40006592  0.099643028
##  [8,]  0.04088587  0.301765840
##  [9,] -0.33862466 -0.001668644
## [10,]  0.22208321  0.042416406
```

```
plot(cmds$points[,1:2])
```




```
plot(eB$vectors[,1:2])
```




Se pueden observar en los anteriores gráficos, los coordenadas son prácticamente iguales.

# 18 Códigos extras

```
load("datos.Rdata")
```

En el archivo "datos.Rdata" se cuenta con la información de tasa de asesinatos y tasa de asaltos de 10 ciudades de los Estados Unidos en 1970. Se busca hacer dos grupos usando el método de k-medias y se inicia con las asignaciones dadas en la base llamada crimen1.

## 18.1 Centroides nuevos a partir de una base de datos

```
head(crimen1)
```

```
##             asesinato asalto grupo
## Anaheim           2.7    103     2
## Baltimore        13.2    396     1
## Boston            4.4     95     1
## Buffalo           5.7    111     1
## Chicago          12.9    233     2
## Cincinnatti       6.4    107     1
```

```
cent1<-apply(crimen1[,1:2],2,tapply,as.factor(crimen1$grupo),mean) # se calculan los nuevos centroides
datosc1<-rbind(cent1,crimen1[,1:2]) # Se colocan los centroides en la base de datos
dmat1<-as.matrix(dist(datosc1)) # Se calculan las distancia de las ciudades a los centroides
dmat1[1:3,] #observe que las 2 primeras filas son los centroides, así como las dos primeras columnas
```

```
##                1        2   Anaheim Baltimore     Boston   Buffalo   Chicago
## 1        0.00000 51.58916 104.93153  188.4373 112.787972 96.738659  25.47362
## 2       51.58916  0.00000  53.50493  240.0209  61.259566 45.208161  77.05334
## Anaheim 104.93153 53.50493   0.00000  293.1881   8.178631  8.544004 130.39954
##         Cincinnatti Cleveland   Dallas   Detroit  Houston
## 1         100.694933  75.63557 130.6170  15.78054 25.18258
## 2          49.134521  24.41211 182.1922  67.16232 27.85949
## Anaheim     5.448853  31.30878 235.5239 120.59851 81.25048
```

```
dg1=dmat1[3:12,1:2] # se extraen las disntancias con los centroides para cada ciudad
(grupo2<-apply(dg1,1,which.min)) # Se ubica cuál es la columna con menor distancia (el mejor centroide)
```

```
##     Anaheim   Baltimore      Boston     Buffalo     Chicago Cincinnatti
##           2           1           2           2           1           2
##   Cleveland      Dallas     Detroit     Houston
##           2           1           1           1
```

## 18.2 Evaluar el agrupamiento a partir de la SCDG

**Se realiza un nuevo agrupamiento**

```
cent2<-apply(crimen1[,1:2],2,tapply,as.factor(grupo2),mean)
datosc2<-rbind(cent2,crimen1[,1:2])
dmat2<-as.matrix(dist(datosc2))
dg2=dmat2[3:12,1:2]
```

```
dc1=c();dc2=c()
for (i in 1:nrow(dg1)){
  dc1[i]=dg1[i,crimen1$grupo[i]] # se obtiene la distancia del grupo asignado
  }
for (i in 1:nrow(dg2)){
  dc2[i]=dg2[i,grupo2[i]]
  }
sc1=sum(dc1^2);sc2=sum(dc2^2) # Se elevan al cuadrado las distancias
sc1;sc2 # Observe que la segunda es más pequeña por lo que la nueva agrupación es mejor
```

```
## [1] 95067.53
```

```
## [1] 32414.28
```

## 18.3 K-means de forma manual

```
# PASO 1: Cálculo de los nuevos centroides

# Se inicializa una matriz vacía de 2 filas (para 2 grupos/clusters)
# y 5 columnas (correspondientes a las primeras 5 variables del dataset)
centros1 = matrix(nrow = 2, ncol = 5)

# Se itera por cada una de las 5 variables numéricas (columnas 1 a 5)
# tapply calcula la media de la variable j separada por la pertenencia a 'grupo'
for (j in 1:5) {
  centros1[, j] = tapply(senti1[, j], senti1$grupo, mean)
}

# PASO 2: Reasignación de observaciones al centroide más cercano
```

```r
# Vector vacío para almacenar la nueva asignación de cluster de cada observación
grupo1 = c()

# Iterar sobre cada fila (observación) del conjunto de datos
for (i in 1:nrow(senti1)) {

  # Vector temporal para almacenar las distancias del punto i a cada centroide
  disti = c()

  # Iterar sobre los dos centroides calculados previamente
  for (j in 1:2) {
    # Se calcula la distancia euclidiana entre el centroide j (centros1[j, ])
    # y la observación i (excluyendo la columna 6 que contiene la etiqueta 'grupo')
    disti[j] = dist(rbind(centros1[j, ], senti1[i, -6]))
  }

  # Se asigna la observación al índice del centroide que tenga la menor distancia
  grupo1[i] = which(disti == min(disti))
}

# PASO 3: Matriz de confusión / Evaluación de convergencia

# Tabla cruzada entre la asignación anterior (senti1$grupo) y la nueva (grupo1)
# Permite verificar cuántas observaciones cambiaron de cluster en esta iteración
table(senti1$grupo, grupo1)

# Forma de ubicar cuáles son lo que se cambiaron de grupo
nums = which(grupo1 != grupo)
base3[nums, c("grupo","grupo1")]
```

```r
# PARTE 1: Cálculo de los CENTROIDES (vector de medias por grupo)
# Centroide del Grupo 1: media de cada una de las variables (columnas 1 a 5)
# Se filtra el subconjunto de filas con grupo == 1 y se excluye la columna 6 ('grupo')
cen1 = colMeans(senti1[senti1$grupo == 1, -6])

# Centroide del Grupo 2: media de cada una de las variables para el grupo 2
cen2 = colMeans(senti1[senti1$grupo == 2, -6])

# PARTE 2: Cálculo del MEDOIDE (observación más central del grupo)
# Subconjunto de datos correspondiente únicamente a las observaciones del Grupo 1
base1 = senti1[senti1$grupo == 1, -6]
# dist(base1) calcula la matriz de distancias por pares entre todas las observaciones del grupo 1
# as.matrix() la convierte a matriz cuadrada simétrica
# colMeans() calcula la distancia promedio de cada observación hacia todas las demás
# min() devuelve el valor numérico de la menor distancia promedio encontrada
min(colMeans(as.matrix(dist(base1))))
```

## 18.4 Cálculo de medoides aplicado

A continuación, se da matriz de distancias para 9 los individuos que se han agrupado en 2 grupos conformados de la siguiente manera: los primeros 4 individuos en el primer grupo y los últimos 5 en el segundo grupo.

```r
d
```

```
##          1     2     3     4     5     6     7     8     9
```

```
##  [1,] 0.000 0.190 0.262 0.310 0.274 0.190 0.274 0.298 0.250
##  [2,] 0.190 0.000 0.119 0.167 0.131 0.000 0.083 0.155 0.107
##  [3,] 0.262 0.119 0.000 0.048 0.155 0.119 0.202 0.131 0.179
##  [4,] 0.310 0.167 0.048 0.000 0.202 0.167 0.250 0.179 0.226
##  [5,] 0.274 0.131 0.155 0.202 0.000 0.131 0.048 0.024 0.190
##  [6,] 0.190 0.000 0.119 0.167 0.131 0.000 0.083 0.155 0.107
##  [7,] 0.274 0.083 0.202 0.250 0.048 0.083 0.000 0.071 0.190
##  [8,] 0.298 0.155 0.131 0.179 0.024 0.155 0.071 0.000 0.214
##  [9,] 0.250 0.107 0.179 0.226 0.190 0.107 0.190 0.214 0.000
```

**Grupo 1**

RECUERDE QUE EL MEDOIDE ES EL PUNTO MÁS BAJO

Son los primeros 4 individuos

```
apply(d[1:4,1:4], 2, \(x) mean(x[x != 0])) # Este código es para no incluir a el cero en el promedio
```

```
##         1         2         3         4
## 0.2540000 0.1586667 0.1430000 0.1750000
```

Medoide: punto 3

**Grupo 1**

Son los 5 individuos restantes

```
apply(d[5:9,5:9], 2, \(x) mean(x[x != 0])) # Este código es para no incluir a el cero en el promedio
```

```
##       5       6       7       8       9
## 0.09825 0.11900 0.09800 0.11600 0.17525
```

Medoide: Punto 7

Luego se saca las distancias de estos grupos con respecto a estos dos medoides, ya que el que esté más cerca será el grupo que se le asigne otra vez

```
cbind(c(3,7),d[c(3,7),-c(3,7)])
```

```
##            1     2     4     5     6     8     9
## [1,] 3 0.262 0.119 0.048 0.155 0.119 0.131 0.179
## [2,] 7 0.274 0.083 0.250 0.048 0.083 0.071 0.190
```

Viendo las distancias y según la distancia más cercana de los demás individuos con respecto a los medoides 3 y 7: El individuo 1 sigue en el grupo 1 El individuo 2 cambia al grupo 2 El individuo 4 sigue en el grupo 1 El individuo 5 sigue en el grupo 2 El individuo 6 sigue en el grupo 2 El individuo 8 sigue en el grupo 2 El individuo 9 cambia al grupo 1

Ahora los grupos son: Grupo 1: 1, 3, 4 y 9 Grupo 2: 2, 5, 6, 7 y 8

## 18.5 Código para distancia entre grupos de forma manual a partir de las distancias:

```
# Índices de los dos grupos originales según el enunciado:
# Grupo A (primeros 4 individuos): 1, 2, 3, 4
# Grupo B (últimos 5 individuos):  5, 6, 7, 8, 9
gA <- 1:4
gB <- 5:9
```

```
# Submatriz de distancias entre todos los elementos de A y todos los de B (dimensión 4 x 5)
d_cruzada <- d[gA, gB]
```

```r
# 1. SALTO MÍNIMO / VECINO MÁS CERCANO (Single Linkage)
# Definición: La menor distancia entre cualquier punto de A y cualquier punto de B
# D_single(A, B) = min_{i in A, j in B} d(i, j)
dist_single <- min(d_cruzada)
# Resultado: distancia mínima entre ambos clusters
```

```r
# 2. SALTO MÁXIMO / VECINO MÁS LEJANO (Complete Linkage)
# Definición: La mayor distancia entre cualquier punto de A y cualquier punto de B
# D_complete(A, B) = max_{i in A, j in B} d(i, j)

dist_complete <- max(d_cruzada)
# Resultado: distancia máxima entre ambos clusters
```

```r
# 3. SALTO PROMEDIO (Average Linkage / UPGMA)
# Definición: El promedio aritmético de todas las distancias entre pares cruzados
# D_average(A, B) = (1 / (|A| * |B|)) * sum_{i in A} sum_{j in B} d(i, j)
dist_average <- mean(d_cruzada)  # Equivale exactamente a sum(d_cruzada) / (length(gA) * length(gB))
```

```r
# 4. SALTO ENTRE MEDOIDES (Medoid Linkage)
# Definición: La distancia directa entre el medoide del Grupo A y el medoide del Grupo B
# Del análisis previo: Medoide de A = punto 3, Medoide de B = punto 7
med_A <- 3
med_B <- 7
dist_medoides <- d[med_A, med_B]
```

```r
# 5. SALTO POR CENTROIDES (Centroid Linkage / UPGMC)
# Nota teórica: Cuando se dispone de las coordenadas originales X, es ||mean(X_A) - mean(X_B)||^2.
# A partir únicamente de una matriz de distancias D (usando distancias euclidianas al cuadrado):
# D_centroid(A, B) = (1/|A|)*sum_{i in A} d^2(i, cB) ... expresado en términos de distancias por pares:
# D^2(bar{x}_A, bar{x}_B) = (1/|A|*|B|) * sum d^2(i,j) - (1/|A|^2)*sum_{i<j} d^2_A - (1/|B|^2)*sum_{i<j

d2_cruzada <- d[gA, gB]^2
d2_interna_A <- d[gA, gA]^2
d2_interna_B <- d[gB, gB]^2

na <- length(gA)
nb <- length(gB)

# Distancia cuadrática entre centroides derivada desde la matriz D:
dist2_centroide <- (sum(d2_cruzada) / (na * nb)) -
                   (sum(d2_interna_A) / (2 * na^2)) -
                   (sum(d2_interna_B) / (2 * nb^2))

dist_centroide <- sqrt(max(0, dist2_centroide))
```

```r
# 6. DISTANCIA POR INCREMENTO DE VARIANZA / CRITERIO DE WARD
# Definición: Mide el incremento en la suma de cuadrados intra-grupo (Delta WSS) si se unieran ambos gr
# Delta WSS = (|A| * |B| / (|A| + |B|)) * ||bar{x}_A - bar{x}_B||^2

delta_ward <- (na * nb / (na + nb)) * (dist_centroide^2)
```

## 18.6 Suma de cuadrados de grupo al medoide

Usando los grupos originales del ejemplo de arriba

```r
#grupo 1
(scdg1 <- sum(d[3,c(1,2,4)]^2))
```

```
## [1] 0.085109
```

```r
#grupo 2
(scdg2 <- sum(d[7,c(5,6,8,9)]^2))
```

```
## [1] 0.050334
```

## 18.7 Distancias para patrones similares (correlación)

```r
R <- cor(t(covid))
d <- 1-R
```

```r
which(d==min(d[d!=0]), arr.ind = T)
```

```
##        row col
## Italy   31  19
## France  19  31
```

Para hacer lo dendogramas recuerde dar un objeto `as.dist()`

```r
# 2. Seleccionar las dos observaciones parecidas (perfiles con la misma tendencia)
obs1 <- as.numeric(datos["Estudiante_A", ])
obs2 <- as.numeric(datos["Estudiante_B", ])
nombres_vars <- colnames(datos)

# 3. Calcular la métrica de distancia de perfil/tendencia (correlación de Pearson)
# Una correlación cercana a +1 indica que ambas curvas suben y bajan en paralelo
# Distancia de tendencia: d = 1 - r (vale 0 si las tendencias son perfectamente paralelas)
r <- cor(obs1, obs2)
dist_tendencia <- 1 - r
```

```r
dist_tendencia
```

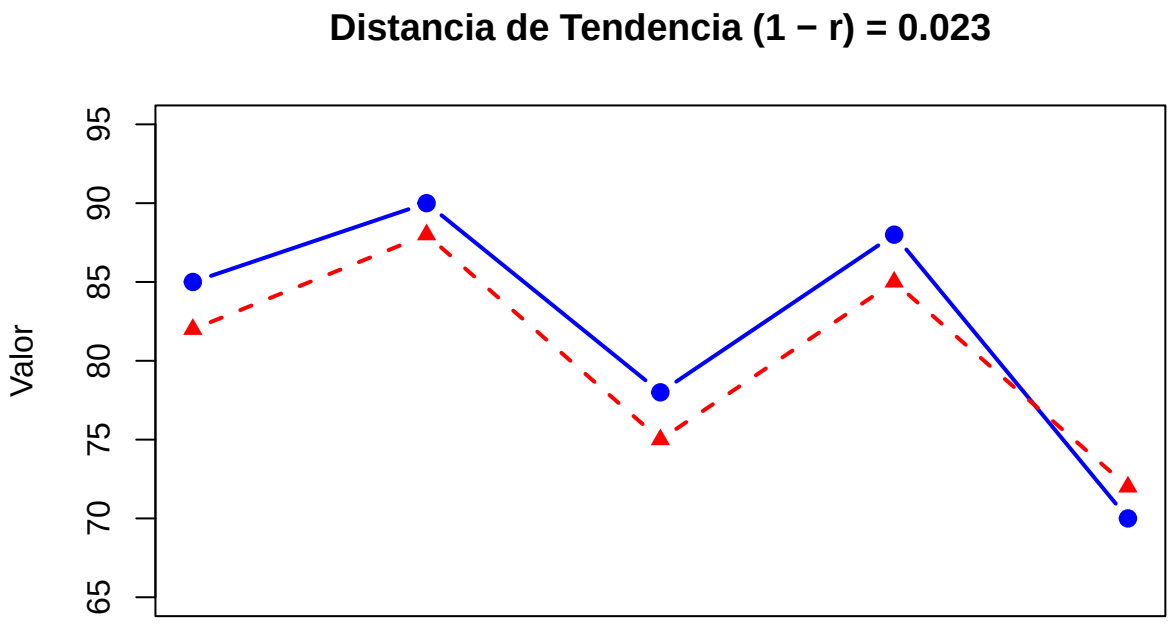
```
## [1] 0.02308894
```

```r
# 4. Gráfico lineal en base R para visualizar la trayectoria o perfil
# Rango común en el eje Y para ambas curvas
rango_y <- range(c(obs1, obs2))

# Trazar la primera observación
plot(
  1:length(obs1), obs1,
  type = "b", pch = 19, col = "blue", lwd = 2,
  xaxt = "n", xlab = "Variables / Materias", ylab = "Valor",
  ylim = c(rango_y[1] - 5, rango_y[2] + 5),
  main = paste("Distancia de Tendencia (1 - r) =", round(dist_tendencia, 3))
)

# Agregar las etiquetas de las variables en el eje X
#axis(side = 1, at = 1:length(obs1), labels = nombres_vars)

# Superponer la segunda observación
lines(1:length(obs2), obs2, type = "b", pch = 17, col = "red", lwd = 2, lty = 2)
```




```
# Cuadrícula para facilitar la lectura visual del paralelismo

#grid()

# Leyenda identificativa
# legend(
#   "bottomleft",
#   legend = c("Estudiante A", "Estudiante B"),
#   col = c("blue", "red"),
#   pch = c(19, 17),
#   lty = c(1, 2),
#   lwd = 2,
#   bty = "n"
# )
```